

A student's guide to the dirac16complex numerical solutions

Installing, deriving, running and checking the five CVODE experiments of Stage 3 on
Windows, macOS and Linux

dirac16complex project (with Claude Opus 5.5)

September 2026

Contents

1	What you will compute and why	5
1.1	The question	5
1.2	The five experiments	6
1.3	What you will have at the end	6
2	How to use this guide	7
2.1	Conventions	7
2.2	Terminals	7
2.3	How this guide was tested	8
3	Installing the tools	8
3.1	What you need	8
3.2	Windows 11	9
3.3	macOS	10
3.4	Linux (Ubuntu or Debian)	10
3.5	The Python packages	11
3.6	Checking the installation	11
4	Getting the code and the solver engine	12
4.1	Choose a short folder	12
4.2	Clone the repository	12
4.3	Fetch the solver engine	13

4.4	Checking the engine	14
4.5	The three engines are not identical	14
4.6	A map of the repository	15
5	Building and testing the program	15
5.1	Build	15
5.2	The FMA note	16
5.3	Test	16
6	The mathematics you need, from zero	17
6.1	Numbers, columns and matrices	17
6.2	The field: a column of 16 complex numbers	17
6.3	Spacetime and the gamma matrices	17
6.4	Three more matrices: C, B and the chirality	19
6.5	Metric, scale factors and the vielbein	19
6.6	The covariant derivative	20
6.7	The Lagrangian and the field equation	20
6.8	From the field equation to a mode equation	21
6.9	Complex to real: the layout that CVODE sees	21
6.10	Energy, pressure, norms and the expectation-value rule	22
6.11	EXP-1: the primordial field	23
6.12	EXP-2: an 8D universe that feels the condensate	24
6.13	EXP-3: the condensate as dark energy	25
6.14	EXP-4: the quanta as dark matter	26
6.15	EXP-5: momentum along an extra time	26
6.16	How CVODE solves $u' = f(t, u)$	27
7	Running the experiments	28
7.1	The command line of the program	28
7.2	A first run into a scratch folder	28
7.3	All five experiments	29
7.4	Reproducing the committed files byte for byte	30

8	Reading the output files	31
8.1	The file format	31
8.2	EXP-1 (folder <code>exp1</code> , 29 files)	31
8.3	EXP-2 (folder <code>exp2</code> , 4 files)	32
8.4	EXP-3 (folder <code>exp3</code> , 10 run files and the analysis)	32
8.5	EXP-4 (folder <code>exp4</code> , 13 files)	33
8.6	EXP-5 (folder <code>exp5</code> , 5 files)	34
9	Checking the results with the Python checkers	35
9.1	What the checkers do	35
9.2	Checking the committed folder (the full form)	35
9.3	Checking a scratch folder (the quick form)	36
10	The Jupyter notebook	36
10.1	What it is	36
10.2	Running it headless	37
10.3	Running it interactively	38
11	The Mathematica notebook (optional)	38
12	Rebuilding the PDF documents (optional)	40
13	Changing parameters: exercises with answers	41
13.1	Tolerances decide whether the checks pass	41
13.2	Convergence	41
13.3	Predicting EXP-1 from the formulas	42
13.4	Solving the EXP-2 constraint yourself	42
13.5	From x_0 to (w_0, w_a)	43
13.6	Moving the EXP-5 turning point	44
13.7	Massless quanta are not created	44
14	Troubleshooting	44
15	Glossary	46

Abstract

This guide takes a student who has never used Rust, CVODE or this repository from an empty computer to a complete and checked reproduction of every numerical solution of Stage 3 of the `dirac16complex` project. Stage 3 consists of five experiments, EXP-1 to EXP-5. Each one integrates the field equation of `dirac16complex`, a 16-component complex Grassmann spinor field in eight dimensions with four space-like and four time-like directions, in a homogeneous background, with the CVODE solver of the pure-Rust SUNDIALS 7.8.0 port of the `rustSolveIt` engine. The guide explains in plain language what is computed and why; how to install Git, Rust, Python with `numpy`, `matplotlib`, `sympy`, `nbformat`, `nbclient` and `ipykernel`, and optionally Jupyter, the Wolfram Engine and a TeX distribution, on Windows 11 (PowerShell and Git Bash), macOS and Linux; how to fetch the code and the pinned solver engine and how to check them; how to build and test the program; the mathematics from zero, including a step-by-step derivation of each experiment's first-order system $u' = f(t, u)$ with its state vector index by index, its right-hand side and its initial data; how CVODE solves such a system; how to run each experiment, read every column of its output and check it with the independent Python checkers (162 checks), the EXP-3 analysis (9 checks), the Jupyter notebook (70 checks) and the Mathematica notebook (49 checks); how to rebuild the PDF documents; exercises with answers; troubleshooting; and a glossary. Every command in this guide was executed while it was written, from fresh clones, on Windows 11 in PowerShell 7 and in Git Bash and on Ubuntu 24.04 under WSL2. Installers, which change the computer, were not executed, and nothing was tested on macOS; Section 2.3 lists exactly what was run where. Two measured facts matter for reproduction: the three platform engines are not byte-identical, and the committed results were produced with the Windows 11 engine, which also builds and reproduces every Rust output byte for byte on Linux.

1 What you will compute and why

1.1 The question

Astronomers infer that most of the energy in the universe is in two forms that are not ordinary matter. **Dark matter** clusters like matter and has almost no pressure. **Dark energy** has a negative pressure and makes the expansion of the universe accelerate. A fluid is described by its energy density ρ and its pressure p , and the single number that tells the two kinds of behaviour apart is the **equation of state parameter**

$$w = \frac{p}{\rho}.$$

Radiation has $w = 1/3$, matter (dust) and dark matter have $w = 0$, and a cosmological constant has $w = -1$. The input document of this study (an e-mail on the Unite supernova compilation, kept by the author and not part of the repository) quotes the Chevallier-Polarski-Linder (CPL) form $w(a) = w_0 + w_a(1 - a)$, where a is the scale factor of the universe ($a = 1$ today), with the Unite values $w_0 = -0.861$ and $w_a = -0.60$, and a constant- w fit $w = -0.764$ about two standard deviations from -1 .

Stage 3 asks: **can the `dirac16complex` field behave like dark matter or like dark energy?** It answers the question with numbers, not words: it solves the field equation numerically in five backgrounds and measures ρ , p and w and how they change.

A caution about the CPL form that the input document gets wrong: its table labels thawing fields " $w_a > 0$ " and freezing fields " $w_a < 0$ ", but its own formula gives $dw/da = -w_a$. A thawing field, whose w rises from -1 as the universe expands, therefore has $w_a < 0$, and a freezing field has $w_a > 0$. This guide always uses the signs that follow from the formula.

1.2 The five experiments

Experiment	Background	What CVOICE integrates	Question
EXP-1	the primordial (pair-creation) field of the author's notebook	one 16-component mode: 32 real ODEs, 26 runs	what do ρ , p and w do in that field, and what source would 8D Einstein gravity need?
EXP-2	homogeneous 8D universe solved together with 8D Einstein gravity	6 geometry ODEs and 32 spinor ODEs, 3 runs	does a self-gravitating condensate isotropise, and what w does a 3-space observer infer?
EXP-3	4D late universe with frozen extra dimensions	time, distance, spinor and $\ln \sigma$: 35 ODEs, 10 runs	can the condensate reproduce the Unite dark energy (w_0, w_a)?
EXP-4	expanding 3-space: radiation era, and de Sitter followed by radiation	32 ODEs per momentum mode, hundreds of modes	do the quanta behave like dark matter, and how many does the expansion create?
EXP-5	deflating extra times	32 ODEs, 4 runs	why must the physics be restricted to modes without momentum along the extra times?

What the committed results say, in one paragraph (the scientific document of Stage 3, DIRAC16COMPLEX DARK SECTOR NUMERICS, discusses them fully): a thermal gas of the free quanta goes from radiation-like, $w = 0.3329$ at $a = 1$, to dust-like, $w = 0.0359$ at $a = 100$, and expansion creates such quanta for $m > 0$ but not for $m = 0$. That is necessary for dark matter, not sufficient. As dark energy the condensate can be tuned to $w_0 = -0.861$ today, but then its tangent slope is $w_a = -4.81$, about eight times faster than Unite, and the model universe bounces at redshift $z = 0.388$. You will reproduce these numbers yourself in Sections 7 to 9.

1.3 What you will have at the end

1. A built program (Section 5.1) that runs the five experiments and checks itself with 69 self-checks.
2. The output files of every experiment, byte for byte identical to the committed ones under `artifacts/dirac16complex/numerics/`.

3. Independent confirmation by five Python checkers that recompute the physics from the raw spinor columns (162 checks), by the EXP-3 analysis (9 checks), by the Jupyter notebook (70 checks and 17 figures) and, optionally, by the Mathematica notebook (49 checks).
4. The understanding to change a parameter, predict the result and check the prediction.

2 How to use this guide

2.1 Conventions

- **Everything is counted from 0.** Coordinates are $x_0, \dots, x_7$, spinor components are $u_0, \dots, u_{15}$, and the first entry of a list is entry 0.
- A box like the ones below contains commands. Type or paste them one line at a time into a terminal and press Enter after each line. The prompt that your terminal prints before the cursor is not shown.
- The sentence before each box says which terminal it is for. **PowerShell** means PowerShell 7 on Windows. **Git Bash** means the bash that comes with Git for Windows. **macOS** and **Linux** mean the Terminal application with bash or zsh. Commands marked "Git Bash, macOS and Linux" are the same in all three.
- **Repository root** means the folder `Dirac_claude` that `git clone` creates; it contains `README.md`, `scripts`, `studies` and `notebooks`. Unless a box starts with a `cd` command, run it from the repository root.
- PowerShell accepts forward slashes in file arguments, so most commands look the same in both shells. Where they differ, both forms are given.
- "Expected output" shows what the command prints when everything works. Numbers of steps and check counts must match exactly; run times depend on your computer.

2.2 Terminals

- **Windows, PowerShell 7:** Start menu, type `pwsh`, press Enter. If `pwsh` is not found, install PowerShell 7 (Section 3.2) or use Windows PowerShell 5.1, called `powershell`, which is part of Windows.
- **Windows, Git Bash:** Start menu, type `Git Bash`, press Enter (installed with Git for Windows).
- **macOS:** Applications, Utilities, Terminal.
- **Linux:** your distribution's terminal (often Ctrl+Alt+T).

To change folder use `cd FOLDER`; `cd ..` goes up one level; `pwd` prints where you are; `ls` lists the folder (these four work in all shells named above).

2.3 How this guide was tested

The commands were executed while the guide was written, from fresh clones of the public repository (Section 4.2) at commit fbec4d7; this guide itself was added later.

Platform	Shells and tools	What was executed
Windows 11 Pro for Workstations 10.0.26200, Intel Core Ultra 9 275HX, 24 cores	PowerShell 7.6.6; Git Bash (GNU bash 5.2.37, Git 2.51.2); rustc and cargo 1.91.1, rustup 1.28.2; Python 3.14.5 with numpy 2.4.6, matplotlib 3.11.0, sympy 1.14.0, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0, jupyterlab 4.4.10, nbconvert 7.16.6; MiKTeX 26.5 (pdfTeX 4.27); WolframScript 1.14.0 with Wolfram 15.0.1	every command of Sections 4 to 13, in both shells where both forms are given
Ubuntu 24.04.4 LTS under WSL2 on the same computer	bash 5.2.21; Git 2.43.0; rustc and cargo 1.93.1; Python 3.12.3 in a virtual environment with numpy 2.5.3, matplotlib 3.11.2, sympy 1.14.0, nbformat 5.11.1, nbclient 0.11.0, ipykernel 7.3.0; TeX Live 2023 (pdfTeX 1.40.25)	clone, engine setup with the <code>linux</code> and <code>win11</code> engines, build, tests, all experiments, all checkers, the analysis, the notebook, a PDF build; no Mathematica (no WolframScript there)
macOS	none	nothing; the macOS commands follow the official instructions of each tool but are untested

Not executed anywhere, because they install software or change system settings: `winget install`, `xcode-select` (macOS), the rustup installer, `apt install`, the MiKTeX, MacTeX and Wolfram installers, and `wolframscript -activate`. On Windows the package identifiers were confirmed with `winget show`, and on Ubuntu the package names with the simulation `apt-get -s install`.

3 Installing the tools

3.1 What you need

Tool	Needed for	Required?
Git	downloading the code and the solver engine; comparing results	yes
Rust (rustup, rustc, cargo)	building and running the five experiments	yes

Tool	Needed for	Required?
a C linker (MSVC Build Tools on Windows, Xcode Command Line Tools on macOS, build-essential on Linux)	Rust uses it to link the program	yes
Python 3.10 or newer	the checkers, the analysis, the notebook, the PDF builder	yes
numpy, matplotlib, sympy, nbformat, nbclient, ipykernel	the checkers and the notebook	yes
JupyterLab (brings nbconvert)	opening the notebook interactively and the nbconvert route	optional
Wolfram Engine or Mathematica (wolframscript)	the Mathematica cross-check notebook	optional
a TeX distribution (MiKTeX, MacTeX or TeX Live)	rebuilding the PDF documents	optional

Section 3.6 shows how to check each of them.

A computer from the last ten years is enough. The program needs only a few megabytes of memory (EXP-4, the largest run, peaked at 14 MB), and the code, the engine, the build and the scratch runs of this guide take about 250 MB of disk.

3.2 Windows 11

The tool `winget` (App Installer) is part of Windows 11. Open PowerShell and install the tools with these commands; each asks for confirmation and some open an installer window:

```
winget install --id Git.Git -e
winget install --id Microsoft.PowerShell -e
winget install --id Rustlang.Rustup -e
winget install --id Python.Python.3.13 -e
```

Rust on Windows needs the Microsoft C++ build tools (the linker `link.exe` and the Windows SDK). If they are not installed, the `rustup` installer offers to install them through the Visual Studio installer; accept that offer. You can also install them yourself:

```
$id = "Microsoft.VisualStudio.2022.BuildTools"
$vs = "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools"
winget install --id $id -e --override "$vs --includeRecommended"
```

Optional tools:

```
winget install --id MiKTeX.MiKTeX -e
winget install --id WolframResearch.WolframEngine -e
```

After installing, **close every terminal and open a new one**, so that the new programs are found. Then make sure the stable Rust toolchain is the default (this is safe to repeat):

```
rustup default stable
```

The Wolfram Engine is free for non-commercial use but must be activated once with a Wolfram ID: run `wolframscript -activate` yourself and follow the prompts. MiKTeX installs missing LaTeX packages on first use; in MiKTeX Console you can allow it to do so without asking.

3.3 macOS

Open Terminal. Install the command line developer tools (they contain Git and the linker that Rust needs):

```
xcode-select --install
```

Install Rust with the official installer, accept the default installation, and load it into the current terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

Install Python 3 from <https://www.python.org/downloads/> (or with Homebrew, `brew install python`). Optional: MacTeX from <https://www.tug.org/mactex/> for the PDFs, and the Wolfram Engine from <https://www.wolfram.com/engine/> for the Mathematica notebook. The Python packages are installed in Section 3.5.

The repository's `.cargo/config.toml` asks the compiler for the x86-64 instruction FMA (Section 5.2). On Apple Silicon (arm64) fused multiply-add is part of the base instruction set. This combination was not tested; if the compiler prints a warning about the target feature `fma`, it is a warning, not an error.

3.4 Linux (Ubuntu or Debian)

Open a terminal and install Git, the compiler tools and Python with its virtual-environment module (you need administrator rights for `sudo`):

```
sudo apt update
sudo apt install -y git curl build-essential python3 python3-venv python3-pip
```

Install Rust exactly as on macOS:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

Optional, for the PDFs:

```
sudo apt install -y texlive-latex-base texlive-latex-recommended lmodern
```

Other distributions have the same tools under similar package names. The Wolfram Engine for Linux is at <https://www.wolfram.com/engine/>.

3.5 The Python packages

Windows (PowerShell or Git Bash). Install the packages for your user:

```
python -m pip install numpy matplotlib sympy nbformat nbclient ipykernel
python -m pip install jupyterlab
```

The second line is optional (interactive notebook and nbconvert). On a computer that already has the packages, pip prints **Requirement already satisfied** for each of them and changes nothing. If pip warns that a script such as `jupyter.exe` is installed in a folder that is not on PATH, this guide avoids the problem by always writing `python -m ...` (Section 14).

macOS and Linux. Recent systems refuse pip install outside a virtual environment (Ubuntu 24.04 prints **error: externally-managed-environment**). Create one virtual environment for this project, outside the repository, and activate it:

```
python3 -m venv ~/.venvs/dirac16
source ~/.venvs/dirac16/bin/activate
python -m pip install numpy matplotlib sympy nbformat nbclient ipykernel
```

Optional: `python -m pip install jupyterlab`. The activation lasts until you close the terminal: **in every new terminal run** `source ~/.venvs/dirac16/bin/activate` again. While it is active, `python` means the Python of the environment, which is why the rest of this guide writes `python` on every platform.

3.6 Checking the installation

In any of the shells:

```
git --version
rustc --version
cargo --version
rustup show active-toolchain
python --version
python -c "import numpy, matplotlib, sympy, nbformat, nbclient, ipykernel; print('OK')"
```

Expected output (your version numbers may be newer):

```
git version 2.51.2.windows.1
rustc 1.91.1 (ed61e7d7e 2025-11-07)
cargo 1.91.1 (ea2d97820 2025-10-10)
stable-x86_64-pc-windows-msvc (default)
Python 3.14.5
OK
```

On Linux the toolchain line is `stable-x86_64-unknown-linux-gnu (default)`. Optional tools:

```
python -m jupyterlab --version
pdflatex --version
wolframscript -version
```

4 Getting the code and the solver engine

4.1 Choose a short folder

On Windows the linker cannot open files whose full path is longer than 260 characters, and the build creates paths about 105 characters below the repository root. Clone into a short folder such as `C:\Users\NAME\src`, not deep inside another folder (a clone at a 277-character path failed with `LNK1104: cannot open file` while this guide was tested). Also avoid folders synchronised by OneDrive or Dropbox, which lock files during the build.

PowerShell:

```
New-Item -ItemType Directory -Force -Path $HOME\src
cd $HOME\src
```

Git Bash, macOS and Linux:

```
mkdir -p ~/src
cd ~/src
```

4.2 Clone the repository

In any shell:

```
git clone https://github.com/once-ere/Dirac_claude.git
cd Dirac_claude
git log -1 --format="%h %s"
```

The last command prints the newest commit. From now on every command runs in this folder, the repository root, unless it says otherwise.

4.3 Fetch the solver engine

The numerical engine is not stored in this repository. A setup script downloads it at a pinned commit into the folder `vendor/rustSolveIt`, which Git ignores. The engine is a pure-Rust translation of SUNDIALS 7.8.0 (the solver library of Lawrence Livermore National Laboratory), published in three platform repositories:

Argument	Repository	Pinned commit
win11	https://github.com/once-ere/rustSolveIt_Win11_SUNDIALS_7_8_0	a8fdff459adfe181573d7924b18bffbd378fdb3
macos	https://github.com/once-ere/rustSolveIt_macos-silicon_SU_NDIALS_7_8_0	5360157f4f6160978f66400566c31b2ae25dd44d
linux	https://github.com/once-ere/rustSolveIt_linux_SUNDIALS_7_8_0	6f58e02e53717a51375bd4bc5918edc57088d922

The script fetches only one of them: the one named by its argument. It makes a shallow (one-commit), sparse checkout of the two folders the study needs, `sundials_rs` (the engine) and `planet_Mercury/notebook` (the notebook templates that the Jupyter notebook was adapted from), about 90 MB. **Use the argument `win11` on every platform** if you want to reproduce the committed results byte for byte; Section 4.5 explains why.

PowerShell:

```
pwsh -NoProfile -File scripts/setup_solver.ps1 -Platform win11
```

If you only have Windows PowerShell 5.1, which blocks scripts by default, allow this one script for this one run:

```
powershell -ExecutionPolicy Bypass -File scripts/setup_solver.ps1 -Platform win11
```

Git Bash, macOS and Linux:

```
bash scripts/setup_solver.sh win11
```

Without an argument, `setup_solver.sh` chooses the platform's own engine (`win11` in Git Bash, `macos` on macOS, `linux` on Linux), and `setup_solver.ps1` defaults to `win11`.

Expected output:

```
solver_platform=win11
solver_commit=a8fdff459adfe181573d7924b18bffbd378fdb3
solver_setup=OK
```

It takes a few seconds. Running it again prints `solver_setup=ALREADY-PRESENT`. If the folder holds a different engine, the script stops with `vendor/rustSolveIt` is at ..., expected ...; remove it and rerun.

4.4 Checking the engine

In any shell:

```
git -C vendor/rustSolveIt rev-parse HEAD
git -C vendor/rustSolveIt sparse-checkout list
```

The first line must print the pinned commit of the table above; the second prints `planet_Mercury/notebook` and `sundials_rs`. The two crates that the study compiles are `vendor/rustSolveIt/sundials_rs/crates/sundials_core` (vectors, matrices, the deterministic mathematical functions) and `vendor/rustSolveIt/sundials_rs/crates/cvode_rs` (the CVODE integrator).

To switch to another engine, delete the folder and run the setup script again. PowerShell:

```
Remove-Item -Recurse -Force vendor/rustSolveIt
```

Git Bash, macOS and Linux:

```
rm -rf vendor/rustSolveIt
```

After a switch, rebuild the program (Section 5).

4.5 The three engines are not identical

The three repositories are often described as vendoring the same engine. They do not, and this was measured while this guide was written. The macOS and Linux engines are identical to each other (their `sundials_rs` folders have the same Git tree, 47d654d5). The Windows 11 engine is different (tree eeaa0cad): its deterministic mathematical library `sundials_libm` is a translation of GNU C Library 2.39, while the other two use different implementations, so some elementary functions can return results that differ in the last bit. The consequences, measured:

1. With the `win11` engine the program reproduces every committed output file byte for byte, on Windows 11 and on Ubuntu 24.04 alike.
2. With the `linux` engine (the default of `setup_solver.sh` on Linux) all 69 self-checks and all 162 checker checks still pass, but 14 committed files differ in their last digits: the ten EXP-1 files with $K = 0$ (backgrounds and $K = 0$ runs) and the three EXP-2 CSV files with their `summary.json`. The same engine gives the same 14 differences on Windows, so they come from the engine, not from the operating system.
3. The Jupyter notebook asserts that fresh outputs are byte-identical to the committed ones, so with the `linux` or `macos` engine its gauntlet stops with `AssertionError('fresh_outputs_byte_identical')`.

The `macos` engine was not run. Because it equals the `linux` engine, it can be expected to behave like it.

4.6 A map of the repository

Folder or file	Contents
<code>studies/dirac16complex_cosmology/</code>	the Rust program: <code>src/exp1.rs</code> to <code>src/exp5.rs</code> (one experiment each), <code>src/spinor.rs</code> (16 by 16 complex algebra), <code>src/driver.rs</code> (the CVODE driver), <code>src/generated.rs</code> (the gamma matrices), <code>src/main.rs</code> (the command line)
<code>scripts/</code>	the setup scripts, the five checkers <code>check_dirac16complex_exp1.py</code> to <code>check_dirac16complex_exp5.py</code> , the EXP-3 analysis <code>analyze_dirac16complex_exp3.py</code> , the PDF builder, the Wolfram scripts
<code>artifacts/dirac16complex/numerics/</code>	the committed outputs: <code>exp1/</code> to <code>exp5/</code> , <code>figures/</code> , <code>numerics-summary.json</code> , <code>notebook-report.json</code> , <code>mathematica-report.json</code>
<code>artifacts/dirac16complex/arbitrary-field/algebra-fixture.json</code>	the exact integer gamma matrices, the source of <code>src/generated.rs</code> and of every checker
<code>notebooks/</code>	the Jupyter notebook <code>dirac16complex_dark_sector.ipynb</code> and its tools, the Mathematica notebook <code>Dirac16ComplexDarkSector.nb</code>
<code>provenance/</code>	the documents in Markdown, LaTeX and PDF, and the registry <code>pdf-specifications.json</code>
<code>wolfram/</code>	the exact Wolfram Language packages of Stages 1 and 2
<code>tests/</code> <code>build/</code> , <code>vendor/</code> , <code>target/</code>	unittest suites of the documents and tools scratch, the engine and the compiler output; all ignored by Git

5 Building and testing the program

5.1 Build

The program is a Rust crate with its own workspace. Build it from its own folder, in release mode (optimised; debug builds are far slower). In any shell:

```
cd studies/dirac16complex_cosmology
cargo build --release
cd ../../
```

Expected output (the first build compiles the engine too and takes a few seconds on a fast computer, a minute or two on a slow one):

```

Compiling sundials_core v7.8.0 (...)
Compiling ccode_rs v7.8.0 (...)
Compiling dirac16complex_cosmology v0.1.0 (...)
Finished ... profile [optimized] target(s) in 4.98s

```

There must be no warning: the crate declares `#![deny(warnings)]`, so any warning would stop the build. The program is now at `studies/dirac16complex_cosmology/target/release/dirac16complex_cosmology.exe` on Windows and at the same path without `.exe` elsewhere.

5.2 The FMA note

The file `.cargo/config.toml` in the repository root contains

```

[build]
rustflags = ["-C", "target-feature=+fma"]

```

Cargo reads it because the crate folder lies below the repository root. It tells the compiler that the processor has the FMA instruction (fused multiply-add: $a \cdot b + c$ computed with a single rounding). The engine's deterministic mathematical functions use fused multiply-add in exactly the places where the reference library does, and with this flag each of them becomes one machine instruction instead of a call into the operating system's C library. The pinned numerical results assume it. Every x86-64 processor since Intel Haswell (2013) and AMD Piledriver (2012) has FMA. On an older processor the program stops at once with an illegal-instruction error; there is no supported workaround that keeps the results identical.

5.3 Test

The crate has 24 unit tests (Clifford relations of the generated gamma matrices, the Hamiltonian, CVODE against exact solutions, closed forms of EXP-2 and EXP-3, the EXP-4 algebra, the output format). In any shell:

```

cd studies/dirac16complex_cosmology
cargo test --release
cd ../..

```

Expected output, among the compiler lines:

```

running 24 tests
test result: ok. 24 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

```

followed by two empty test groups (`running 0 tests`). One test deliberately makes the right-hand side fail and checks that the error is reported, so the line

```

[ERROR] [rank 0] [...ccode.rs:4221] [cvHandleFailure] At t = 0.496781677252198,
the right-hand side routine failed in an unrecoverable manner.

```

(one line on your screen) is expected and is not a failure.

6 The mathematics you need, from zero

This section explains everything the program does, with nothing assumed beyond complex numbers and derivatives. The physics behind the field equation is derived in the documents of Stage 1 (the arbitrary gravitational field) and Stage 2 (the primordial field) in the folder **provenance/**; here we use their results.

6.1 Numbers, columns and matrices

A complex number is $z = x + iy$ with real x, y and $i^2 = -1$; its conjugate is $z^* = x - iy$ and $|z|^2 = z^*z = x^2 + y^2$. A **column** of 16 complex numbers is written

$$u = \begin{pmatrix} u_0 \\ u_1 \\ \vdots \\ u_{15} \end{pmatrix}, \quad u^\dagger = (u_0^*, u_1^*, \dots, u_{15}^*).$$

The row $u^\dagger$ (read "u dagger") is the conjugate transpose. A 16 by 16 matrix M acts on u by $(Mu)_i = \sum_{j=0}^{15} M_{ij}u_j$, where i is the row and j the column, both counted from 0. The product $u^\dagger Mu = \sum_{i,j} u_i^* M_{ij}u_j$ is a single number; for $M = 1$ it is $u^\dagger u = \sum_i |u_i|^2$, the squared length of u . A matrix is **Hermitian** if $M^\dagger = M$; then $u^\dagger Mu$ is real. A matrix whose every row contains exactly one nonzero entry, equal to $+1$ or -1 , is a **signed permutation**: it only reorders the components of u and flips some signs.

6.2 The field: a column of 16 complex numbers

The field `dirac16complex` is $\Psi = (\Psi_0, \dots, \Psi_{15})^T$: at every point of spacetime, 16 complex components. In the quantum theory the components are Grassmann-odd, which means $\Psi_a \Psi_b = -\Psi_b \Psi_a$; this is how fermions obey the Pauli principle (a state holds at most one particle). None of the ODEs below multiplies two Grassmann numbers. Every experiment integrates the **mode amplitude** $u(t)$, an ordinary column of 16 complex numbers, and the Grassmann nature enters only in two places: the counting of states (each state occupied at most once, Section 6.15) and the expectation-value rule of Section 6.10. This is the mean-field, one-particle reading of the field; it is exact for the free modes and an approximation for the self-interacting condensate.

6.3 Spacetime and the gamma matrices

Spacetime has eight coordinates $x_0, \dots, x_7$. The flat metric is

$$\eta = \text{diag}(+1, +1, +1, +1, -1, -1, -1, -1) :$$

directions 0 to 3 are space-like, directions 4 to 7 time-like. x_0 is a hidden space direction, x_1, x_2, x_3 are ordinary 3-space, x_4 is **the** time t , and x_5, x_6, x_7 are three extra time directions. We write $T = \{0, 1, 2, 3, 5, 6, 7\}$ for the seven directions other than time.

The eight **gamma matrices** $\gamma^0, \dots, \gamma^7$ are real 16 by 16 matrices with integer entries (the author's notebook calls them T16). They satisfy the Clifford relation

$$\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab} \cdot 1,$$

so $(\gamma^a)^2 = +1$ for $a = 0, \dots, 3$, $(\gamma^a)^2 = -1$ for $a = 4, \dots, 7$, and two different gamma matrices anticommute. Each of them is a signed permutation, which makes it easy to write down completely. The table gives, for each row i , the component that γ^a picks and its sign: the entry -13 in row 0 of γ^4 means $(\gamma^4 u)_0 = -u_{13}$, and $+0$ means $+u_0$.

i	γ^0	γ^1	γ^2	γ^3
0	+8	-15	+14	-13
1	+9	-14	-15	+12
2	+10	+13	-12	-15
3	+11	+12	+13	+14
4	+12	-11	+10	-9
5	+13	-10	-11	+8
6	+14	+9	-8	-11
7	+15	+8	+9	+10
8	+0	+7	-6	+5
9	+1	+6	+7	-4
10	+2	-5	+4	+7
11	+3	-4	-5	-6
12	+4	+3	-2	+1
13	+5	+2	+3	-0
14	+6	-1	+0	+3
15	+7	-0	-1	-2

i	γ^4	γ^5	γ^6	γ^7
0	-13	+14	+15	+8
1	+12	+15	-14	+9
2	+15	-12	+13	+10
3	-14	-13	-12	+11
4	+9	-10	-11	-12
5	-8	-11	+10	-13
6	-11	+8	-9	-14
7	+10	+9	+8	-15
8	+5	-6	-7	-0
9	-4	-7	+6	-1
10	-7	+4	-5	-2
11	+6	+5	+4	-3
12	-1	+2	+3	+4
13	+0	+3	-2	+5
14	+3	-0	+1	+6
15	-2	-1	-0	+7

The same matrices are stored exactly in `artifacts/dirac16complex/arbitrary-field/algebra-fixture.json` (key `gamma`, zero-based) and in `studies/dirac16complex_cosmology/src/generated.rs`. $\gamma^0, \dots, \gamma^3$ are symmetric and $\gamma^4, \dots, \gamma^7$ antisymmetric.

A check you can do by hand. Apply γ^4 twice to component 0: $(\gamma^4 u)_0 = -u_{13}$, and row 13 of γ^4 says $(\gamma^4 v)_{13} = +v_0$, so $(\gamma^4 \gamma^4 u)_0 = -(\gamma^4 u)_{13} = -u_0$. That is $(\gamma^4)^2 = -1$ in row 0, as the Clifford relation demands for a time-like direction.

6.4 Three more matrices: C, B and the chirality

- The **charge matrix** $C = \gamma^0 \gamma^1 \gamma^2 \gamma^3$ is real and symmetric with $C^2 = 1$. The **Dirac adjoint** is $\bar{\Psi} = \Psi^\dagger C$, and $S = \bar{\Psi} \Psi$ is the scalar density (the condensate).
- $B = -iC\gamma^4$ is Hermitian with $B^2 = 1$; it has the eigenvalue $+1$ eight times and -1 eight times. It defines the indefinite inner product $u^\dagger B u$ (Section 6.10).
- The **chirality** $\gamma^8 = \gamma^0 \gamma^1 \dots \gamma^7 = \text{diag}(-1, \dots, -1, +1, \dots, +1)$: components 0 to 7 have chirality -1 , components 8 to 15 chirality $+1$.

C is a signed permutation and B is i times a signed permutation:

i	C	B/i
0	-4	+9
1	-5	-8
2	-6	-11
3	-7	+10
4	-0	-13
5	-1	+12
6	-2	+15
7	-3	-14
8	+12	+1
9	+13	-0
10	+14	-3
11	+15	+2
12	+8	-5
13	+9	+4
14	+10	+7
15	+11	-6

So, for example, $(Bu)_0 = i u_9$.

6.5 Metric, scale factors and the vielbein

All five backgrounds are homogeneous: nothing depends on position, only on time. With $t = x_4$ their line element is

$$ds^2 = -dt^2 + \sum_{j \in T} \epsilon_j h_j(t)^2 dx_j^2, \quad \epsilon_j = +1 \ (j = 0, 1, 2, 3), \quad \epsilon_j = -1 \ (j = 5, 6, 7).$$

$h_j(t)$ is the **scale factor** of direction j : a distance dx_j along it is physically $h_j dx_j$ long. Its **expansion rate** is $H_j = \dot{h}_j/h_j$ (a dot is d/dt), the total rate is $\Theta = \sum_{j \in T} H_j$ and the 7-volume

is $V = \prod_{j \in T} h_j$, so that $\dot{V}/V = \Theta$. The experiments group the directions: $b = h_0$ (hidden space), $a = h_1 = h_2 = h_3$ (3-space), $c = h_5 = h_6 = h_7$ (extra times), and then $\Theta = H_b + 3H_a + 3H_c$ and $V = b a^3 c^3$.

The **vielbein** (German "eight legs") is the set of eight 1-forms $e^4 = dt$ and $e^j = h_j dx_j$; it turns the curved metric into the flat one, $ds^2 = \eta_{ab} e^a e^b$. The curved gamma matrices are $\gamma^{x_4} = \gamma^4$ and $\gamma^{x_j} = \gamma^j/h_j$: the frame matrix divided by the scale factor.

6.6 The covariant derivative

A spinor cannot be differentiated with the plain derivative $\partial_\mu = \partial/\partial x_\mu$ in a curved spacetime, because the frame in which its components are measured turns from point to point. The **covariant derivative** adds a correction, the spinor connection Ω_μ :

$$D_\mu \Psi = \partial_\mu \Psi + \Omega_\mu \Psi, \quad \Omega_\mu = \frac{1}{2} \omega_{\mu ab} S^{ab}, \quad S^{ab} = \frac{1}{4} (\gamma^a \gamma^b - \gamma^b \gamma^a),$$

summed over all ordered pairs (a, b) . The spin connection $\omega_{\mu ab}$ is fixed by the requirement that the total covariant derivative of the vielbein vanishes (the vielbein postulate); Stage 1 derives and verifies this exactly. The field equation needs only the combination $\gamma^\mu \Omega_\mu$ (summed over μ), and for a diagonal vielbein $e^b = h_b dx_b$ Stage 1 proves the formula

$$\gamma^\mu \Omega_\mu = \frac{1}{2} \sum_{b=0}^7 \frac{1}{h_b} \partial_b \ln \left(\prod_{c \neq b} h_c \right) \gamma^b.$$

In our backgrounds only $b = 4$ has a nonzero derivative ($h_4 = 1$, $\partial_4 = d/dt$), and $\prod_{c \neq 4} h_c = V$, so

$$\gamma^\mu \Omega_\mu = \frac{1}{2} \frac{d \ln V}{dt} \gamma^4 = \frac{1}{2} \Theta \gamma^4.$$

That one line is all of the curved-space geometry that enters EXP-2 to EXP-5.

6.7 The Lagrangian and the field equation

The Lagrangian density of dirac16complex is

$$\mathcal{L} = \sqrt{|g|} \left[\frac{1}{2} (\bar{\Psi} \gamma^\mu D_\mu \Psi - (D_\mu \bar{\Psi}) \gamma^\mu \Psi) - m \bar{\Psi} \Psi - U(\bar{\Psi} \Psi) \right], \quad U(S) = \frac{\lambda}{2} S^2,$$

with mass m and a four-fermion self-interaction of strength λ ($\lambda < 0$ is attractive). Varying it with respect to $\bar{\Psi}$ gives the field equation

$$\gamma^\mu D_\mu \Psi = M_{\text{eff}} \Psi, \quad M_{\text{eff}} = m + U'(S) = m + \lambda S.$$

M_{eff} is the effective mass: the self-interaction shifts the mass in proportion to the condensate S .

6.8 From the field equation to a mode equation

Take a plane wave with coordinate momentum k_j along the transverse directions,

$$\Psi = V^{-1/2} e^{i \sum_j k_j x_j} u(t).$$

Insert it into the field equation, one term at a time.

1. Time derivative: $\partial_t \Psi = V^{-1/2} e^{ik \cdot x} (\dot{u} - \frac{1}{2} \Theta u)$, because $\dot{V}/V = \Theta$.
2. Space derivatives: $\partial_j \Psi = ik_j \Psi$, and the curved gamma is γ^j/h_j , so these terms give $i \sum_j (k_j/h_j) \gamma^j u$ (times the common factor $V^{-1/2} e^{ik \cdot x}$).
3. Connection: $\gamma^\mu \Omega_\mu \Psi = \frac{1}{2} \Theta \gamma^4 \Psi$.

Adding them, the two Θ terms cancel exactly: that is why the factor $V^{-1/2}$ was put in front. What remains is

$$\gamma^4 \dot{u} + i \sum_{j \in T} \frac{k_j}{h_j} \gamma^j u = M_{\text{eff}} u.$$

Multiply from the left by γ^4 and use $(\gamma^4)^2 = -1$: $-\dot{u} + i \sum_j (k_j/h_j) \gamma^4 \gamma^j u = M_{\text{eff}} \gamma^4 u$. Multiply by $-i$ and rearrange:

$$i \dot{u} = h(t) u, \quad h(t) = -i M_{\text{eff}} \gamma^4 - \gamma^4 \sum_{j \in T} \frac{k_j}{h_j} \gamma^j.$$

This is a Schrödinger-type equation with the **mode Hamiltonian** $h(t)$, a 16 by 16 complex matrix. Two facts about it are used everywhere. First, $h^2 = E^2 \cdot 1$ with

$$E^2 = M_{\text{eff}}^2 + \sum_{j=0}^3 \left(\frac{k_j}{h_j} \right)^2 - \sum_{j=5}^7 \left(\frac{k_j}{h_j} \right)^2,$$

so the energies are $+E$ and $-E$, eight times each. Second, h is Hermitian exactly when $k_5 = k_6 = k_7 = 0$. This is **the good sector**: no momentum along the extra times. Then E is real and $u^\dagger u$ is conserved. With extra-time momentum E^2 can become negative, which EXP-5 studies.

6.9 Complex to real: the layout that CVODE sees

CVODE integrates real numbers. Write $u = x + iy$ with real columns x, y and $h = h_r + ih_i$ with real matrices h_r, h_i . Then $i(\dot{x} + i\dot{y}) = (h_r + ih_i)(x + iy)$; the imaginary part and the real part of this equation give

$$\dot{x} = h_r y + h_i x, \quad \dot{y} = h_i y - h_r x.$$

The program stores the spinor as 32 consecutive reals, the **state layout** used by every experiment:

State index	Quantity	CSV column
$0, \dots, 15$	$x_n = \text{Re } u_n, n = 0, \dots, 15$	u_re_0 to u_re_15
$16, \dots, 31$	$y_n = \text{Im } u_n, n = 0, \dots, 15$	u_im_0 to u_im_15

For $h = -iM\gamma^4 - \sum_j (k_j/h_j)\gamma^4\gamma^j$ the pieces are $h_i = -M\gamma^4$ and $h_r = -\sum_j (k_j/h_j)\gamma^4\gamma^j$. **A worked component.** For $k = 0$ ($h_r = 0$) the equation for state index 0 is $\dot{x}_0 = (h_i x)_0 = -M(\gamma^4 x)_0 = -M(-x_{13}) = Mx_{13}$: one multiplication. The whole right-hand side of a mode is 32 such terms plus, when $k \neq 0$, the h_r terms.

6.10 Energy, pressure, norms and the expectation-value rule

Quantising the field with respect to t gives the anticommutator $\{\Psi, \Psi^\dagger\} \propto B$. Because B has eigenvalues of both signs, the space of states carries an indefinite inner product (a Krein space). The experiments use its positive-norm quantisation, in which a one-particle state built on a normalised mode u ($u^\dagger u = 1$) has the **expectation-value rule**

$$\langle \Psi^\dagger M \Psi \rangle = u^\dagger B M u.$$

From it follow the three quantities per mode that every experiment computes:

$$s(u) = u^\dagger B C u = u^\dagger (-i\gamma^4) u, \quad \varepsilon(u) = u^\dagger h u, \quad p_j(u) = -\frac{k_j}{h_j} u^\dagger \gamma^4 \gamma^j u \quad (\text{no sum}),$$

the scalar density, the energy and the pressure along direction j (one uses $BC = -i\gamma^4$, since C commutes with γ^4 and $C^2 = 1$). They satisfy $\varepsilon = M_{\text{eff}} s + \sum_j p_j$. Two norms are monitored:

- the **Hilbert norm** $u^\dagger u$, conserved when h is Hermitian (good sector);
- the **Krein norm** $u^\dagger B u$, conserved always, because $h^\dagger B = B h$.

The rest eigenvector. For $k = 0$, $h = -iM\gamma^4$, and a positive-energy eigenvector obeys $-i\gamma^4 u = u$. The program's standard choice, also an eigenvector of B with eigenvalue +1, is

$$u_0 = \frac{1}{2}(e_0 - e_4 - i e_9 - i e_{13}),$$

where e_n is the column with 1 in position n . Check it with the tables: $(\gamma^4 u_0)_0 = -u_{13} = i/2$, so $(-i\gamma^4 u_0)_0 = 1/2 = (u_0)_0$; do the same for components 4, 9 and 13, and for $(B u_0)_0 = i u_9 = 1/2$. Hence $s(u_0) = u_0^\dagger u_0 = 1$ and $h u_0 = M u_0$.

Energy density, pressure and w . With n modes per unit volume, the energy density is $\rho = n\varepsilon - (\lambda/2)S^2$ and the pressure along j is $p_j = n p_j(u) + (\lambda/2)S^2$; the mean transverse pressure is $\bar{p} = \frac{1}{7} \sum_{j \in T} p_j$ and $w = \bar{p}/\rho$. For a condensate at rest this gives $\rho = mS + \frac{\lambda}{2}S^2$ and $p = \frac{\lambda}{2}S^2$.

Kinetic and potential energy. The input document describes a scalar field with $\rho = \frac{1}{2}\dot{\phi}^2 + V$ and $p = \frac{1}{2}\dot{\phi}^2 - V$. The spinor analogues, both reported by every experiment, are the Lagrangian split $KE_L = \frac{1}{2}n\varepsilon$, $PE_L = \rho - KE_L$ (for a condensate at rest $\rho = KE_L + PE_L$ and $p = KE_L - PE_L$ exactly as for the scalar field, and $w < -1$ with $\rho > 0$ happens exactly when $KE_L < 0$) and the Hamiltonian split $PE_H = mS + U(S)$ (rest mass and interaction), $KE_H = \rho - PE_H$ (momentum energy). For a free gas $KE_L = PE_L = \rho/2$ in every mode, so the scalar-field relation $p = KE - PE$ holds only for condensates, not for a gas.

6.11 EXP-1: the primordial field

Background. The author's notebook field (Stage 2) in the warped coordinate $\zeta = \ln(\sin z)/(6H)$, $z = 6Hx_0$, with $H = 1$ and $t = x_4$:

$$ds^2 = d\zeta^2 - dt^2 + e^{2H\zeta} [e^{2a_4(t)}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4(t)}(dx_5^2 + dx_6^2 + dx_7^2)].$$

3-space grows as e^{a_4} and the extra times shrink as e^{-a_4} , so the 7-volume stays constant. The profile is a primordial window, $a_4'(t) = A(1 + \tanh \frac{t-t_1}{\Delta})(1 - \tanh \frac{t-t_2}{\Delta})/4$ with $t_1 = 2$, $t_2 = 7$, $\Delta = 0.5$ and two amplitudes $A = 1$ and $A = 2$.

Reduction, step by step. Here the vielbein depends on ζ and t , so use the formula of Section 6.6 with $b = 0$ (the ζ direction, $h_0 = 1$) and $b = 4$. For $b = 0$: $\prod_{c \neq 0} h_c = e^{3(H\zeta + a_4)} e^{3(H\zeta - a_4)} = e^{6H\zeta}$, whose ζ -derivative of the logarithm is $6H$; for $b = 4$: $\prod_{c \neq 4} h_c = e^{6H\zeta}$ does not depend on t . Hence $\gamma^\mu \Omega_\mu = 3H\gamma^0$, and a_4 has dropped out. Take a wave in the hidden direction and no momentum elsewhere, $\Psi = e^{-3H\zeta} e^{iK\zeta} u(t)$. The ζ -derivative gives $\gamma^0(-3H + iK)\Psi$; the $-3H$ cancels the connection term $3H\gamma^0\Psi$ (the factor $e^{-3H\zeta}$ plays the role of $V^{-1/2}$), and what remains is

$$\gamma^4 \dot{u} = (M_{\text{eff}} - iK\gamma^0)u \quad \Longleftrightarrow \quad i\dot{u} = hu, \quad h = -iM_{\text{eff}}\gamma^4 - K\gamma^4\gamma^0.$$

This is Section 6.8 with $k_0/h_0 = K$. Stage 2 verified the reduction exactly (Wolfram check `P_modes_exactReduction`). Because h does not contain a_4 , the two profiles must give identical spinors; they do, bit for bit.

State, right-hand side, initial data. The state is the 32-real spinor of Section 6.9, with $h_i = -M_{\text{eff}}\gamma^4$ and $h_r = -K\gamma^4\gamma^0$; $M_{\text{eff}} = m + \lambda S_0 s(u)$ with $m = 1$. Runs: $K \in \{0, 0.5, 2\}$, four initial spinors each, $\lambda = 0$ and $S_0 = 1$, for each profile ($2 \times 3 \times 4 = 24$ runs), plus one run per profile with $\lambda = 0.5$, $S_0 = 1$, $K = 0.5$ (26 runs). The four initial spinors are the first vector of the joint eigenspace of h (energy $+E$) and B ($+1$), called `pos_Bp`; energy $+E$ with $B = -1$ (`pos_Bm`); energy $-E$ with $B = +1$ (`neg_Bp`); and a normalised generic vector `mix` with $\text{Re } u_n = 1/(1+n)$ and $\text{Im } u_n = 0.1(n^2 \bmod 7) - 0.3$ before normalisation. Eigenstates test that ρ and p are frozen; the mixture of both energy signs tests interference. The λ run starts on the self-consistent eigenvector with $M_* = m + \lambda S_0 M_*/\sqrt{M_*^2 + K^2}$, solved by Newton's method: $M_* = 1.4734826640642023$. Time runs from 0 to 10 with 201 samples.

Exact solution and checks. With $\lambda = 0$, h is constant and $u(t) = (\cos Et - i \sin Et h/E) u(0)$ with $E = \sqrt{M^2 + K^2}$; the program compares with it at every sample. It also records the source that 8D Einstein gravity would need for this field ($\kappa = 1$): $\rho_{\text{req}} = -3H^2(7 + a_4'^2)$, which is negative always.

6.12 EXP-2: an 8D universe that feels the condensate

Background and Einstein equations. $ds^2 = -dt^2 + b^2 dx_0^2 + a^2(dx_1^2 + dx_2^2 + dx_3^2) - c^2(dx_5^2 + dx_6^2 + dx_7^2)$ with 8D gravitational coupling $\kappa = 1$ and $m = 1$. For this metric the time-time component of Einstein's equation $G^\mu{}_\nu = \kappa T^\mu{}_\nu$ is a constraint, and the space-space components, after the trace is used, are evolution equations (the signs ϵ_j drop out of the mixed components):

$$\sum_{i < j} H_i H_j = 3H_b H_a + 3H_b H_c + 3H_a^2 + 3H_c^2 + 9H_a H_c = \kappa \rho, \quad \dot{H}_i = -H_i \Theta + \frac{\kappa(\rho - p)}{6}.$$

The sum runs over the 21 pairs of the seven transverse directions; the 6 is $D - 2$ for $D = 8$. The checker derives the Einstein tensor again from the metric, independently (check `einsteinTensorFromMetric`).

Matter. The condensate is the $k = 0$ mode: $h = -iM_{\text{eff}}\gamma^4$, $S = S_0 (V_0/V) s(u)$ with $V_0 = 1$, $M_{\text{eff}} = m + \lambda S$, $\rho = mS + \frac{\lambda}{2}S^2$, $p = \frac{\lambda}{2}S^2$, so $\rho - p = mS$ does not depend on λ .

State vector (38 reals) and right-hand side:

State index	Quantity	d/dt of it
0	$\ln b$	H_b
1	$\ln a$	H_a
2	$\ln c$	H_c
3	H_b	$-H_b \Theta + \kappa m S/6$
4	H_a	$-H_a \Theta + \kappa m S/6$
5	H_c	$-H_c \Theta + \kappa m S/6$
6 to 21	$\text{Re } u_0, \dots, \text{Re } u_{15}$	$-M_{\text{eff}} \gamma^4 \text{Re } u$
22 to 37	$\text{Im } u_0, \dots, \text{Im } u_{15}$	$-M_{\text{eff}} \gamma^4 \text{Im } u$

with $\Theta = H_b + 3H_a + 3H_c$, $V = \exp(\ln b + 3 \ln a + 3 \ln c)$ and S from the spinor. The spinor rows are Section 6.9 with $h_r = 0$.

Initial data and why. $\ln b = \ln a = \ln c = 0$, $H_b = 0$, $H_a = 1$, $H_c = -0.2$ (3-space expanding, extra times deflating), $u(0) = u_0$ of Section 6.10 (a positive-energy particle at rest, $s = 1$). The parameter $x_0 = \lambda S_0/(2m)$ (not the coordinate) is the interaction strength relative to the mass; runs $x_0 \in \{0, -0.4, +0.5\}$. The constraint must hold at $t = 0$: its left side is $C_0 = 3(1) + 3(0.04) + 9(-0.2) = 1.32$, the right side is $\kappa m S_0(1 + x_0)$, hence

$$S_0 = \frac{1.32}{1 + x_0}, \quad \lambda = \frac{2m x_0}{S_0} :$$

$S_0 = 1.32, 2.2, 0.88$ and $\lambda = 0, -0.363636, 1.136364$ for the three runs. The constraint is not imposed again; the program monitors it (`constraint_relative`).

Exact solution. Because $\rho - p = mS$ and SV is constant, the equations give $\dot{V} = \frac{7}{6}\kappa m S_0$, so $V(t) = 1 + 2.4t + \alpha t^2$ with $\alpha = 7S_0/12$, and $H_i V$ grows linearly. The output times are chosen uniform in $\ln V$ from -7 (just before the backward singularity) to 18.5 (late times, $t \approx 10^4$), step 0.05; CVODE integrates the full 38-dimensional system without using the exact solution.

6.13 EXP-3: the condensate as dark energy

Model. The hidden and extra dimensions are frozen, so the 4D universe is an ordinary flat Friedmann universe. Units $H_0 = c = 1$ (today's expansion rate), densities in units of the critical density $3H_0^2/\kappa_4$. The independent variable is the number of e-folds $N = \ln a$ ($N = 0$ today). Since $S \propto a^{-3}$, write $\sigma = S/S_0 = a^{-3}$ and $x_0 = \lambda S_0/(2m)$ as before. From $\rho = mS + \frac{\lambda}{2}S^2 = mS_0\sigma(1 + x_0\sigma)$, normalised so that $\rho_\psi(a = 1) = \Omega_\psi$:

$$\rho_\psi = A\sigma(1 + x_0\sigma), \quad p_\psi = Ax_0\sigma^2, \quad A = \frac{\Omega_\psi}{1 + x_0}, \quad w = \frac{x_0\sigma}{1 + x_0\sigma},$$

and the expansion rate

$$E^2 = \frac{H^2}{H_0^2} = \Omega_r a^{-4} + \Omega_m a^{-3} + \rho_\psi,$$

with $\Omega_r = 0.00009$, $\Omega_m = 0.305$, $\Omega_\psi = 0.69491$ (they add up to 1). Today $w_0 = x_0/(1 + x_0)$, so $x_0 = w_0/(1 - w_0)$: $w_0 = -0.861$ needs $x_0 = -0.462654$, and $w_0 = -0.764$ needs $x_0 = -0.433107$.

Change of variable. $dN = H dt = H_0 E dt$, so $d/dN = (1/(H_0 E)) d/dt$. The spinor equation $i\dot{u} = hu$ with $h = -iM_{\text{eff}}\gamma^4$ becomes $du/dN = -i(h/H_0)u/E$. The time since today is $H_0(t - t_0)$ with $d(H_0 t)/dN = 1/E$, and the comoving distance obeys $dD_C/dz = 1/E$ with $z = e^{-N} - 1$, hence $dD_C/dN = -1/(aE)$.

State vector (35 reals) and right-hand side:

State index	Quantity	d/dN of it
0	$H_0(t - t_0)$	$1/E$
1	D_C (units c/H_0)	$-1/(aE)$
2 to 17	$\text{Re } u_0, \dots, \text{Re } u_{15}$	$-(M_{\text{eff}}/H_0)\gamma^4 \text{Re } u/E$
18 to 33	$\text{Im } u_0, \dots, \text{Im } u_{15}$	$-(M_{\text{eff}}/H_0)\gamma^4 \text{Im } u/E$
34	$\ln \sigma$	$-3 +$ $2 \text{Re}(u^\dagger(-i\gamma^4) du/dN)/s(u)$

Here $a = e^N$, $\sigma = a^{-3}s(u)/s(u_0)$ is taken from the spinor, and $M_{\text{eff}}/H_0 = \mu(1 + 2x_0\sigma)/(1 + 2x_0)$, where $\mu = M_{\text{eff}}(a = 1)/H_0 \in \{3, 7\}$ is a reduced frequency. Real fermions have $m/H_0 > 10^{30}$; two small values show that ρ , p and w do not depend on the spinor's phase. Row 34 reproduces $d\sigma/dN = -3\sigma$ from the spinor's own scalar density.

Initial data and range. At $N = 0$: $H_0(t - t_0) = 0$, $D_C = 0$, $u = u_0$, $\ln \sigma = 0$. Runs $x_0 \in \{-0.462654, -0.433107, -0.3, -0.2, 0\}$ and $\mu \in \{3, 7\}$ (10 runs), integrated backward towards $a = 1/3.5$ and forward to $a = 2$. For every $x_0 < 0$ the attractive term makes ρ_ψ negative in the past and E^2 reaches 0 (a bounce) before $a = 1/3.5$, at the root a_b of $(\Omega_m + A)a^3 + \Omega_r a^2 + Ax_0 = 0$; the backward run then stops just after the bounce ($N_b + 0.01$). Only $x_0 = 0$ reaches $a = 1/3.5$.

The CPL tangent. $w(a) = x_0/(a^3 + x_0)$, so $dw/da = -3a^2x_0/(a^3 + x_0)^2$ and at $a = 1$

$$w_a = -\left.\frac{dw}{da}\right|_{a=1} = \frac{3x_0}{(1 + x_0)^2}.$$

The EXP-3 analysis (`scripts/analyze_dirac16complex_exp3.py`, numpy only, with its own Nelder-Mead minimiser) computes these tangents, least-squares CPL fits, distance-modulus fits and a fine scan over x_0 from the closed forms, and uses the CVODE D_C as a cross-check.

6.14 EXP-4: the quanta as dark matter

Background. Only 3-space expands, $b = c = 1$, $h_1 = h_2 = h_3 = a(t)$. A mode moving along x_1 with comoving momentum k has physical momentum $K = k/a$ and

$$h = -im\gamma^4 - K\gamma^4\gamma^1, \quad E = \sqrt{m^2 + K^2}, \quad h_i = -m\gamma^4, \quad h_r = -K\gamma^4\gamma^1.$$

Each mode is one 32-real state (Section 6.9); the right-hand side is $\dot{x} = -m\gamma^4x - K\gamma^4\gamma^1y$, $\dot{y} = -m\gamma^4y + K\gamma^4\gamma^1x$.

(a) Thermal gas. Radiation era $a(t) = (t/t_i)^{1/2}$ with $t_i = 10$, so the initial Hubble rate is $H_i = 1/(2t_i) = 0.05$; a runs from 1 to 100 (t from 10 to 10^5), with 61 samples uniform in $\ln a$. Units $m = 1$, initial temperature $T_i = 10$ (relativistic). Momentum grid: 48 Gauss-Legendre nodes k_n on $[0, 120]$ with weights w_n . Each mode starts at t_i on the positive-energy eigenvector of $h(t_i)$ with $B = +1$. Fermi-Dirac occupation $f_n = 1/(e^{E_n/T_i} + 1) \leq 1$ (the Pauli principle), and 16 states per momentum (8 particle and 8 antiparticle states). With $W_n = \frac{16}{2\pi^2} w_n k_n^2 f_n$:

$$\rho = \frac{1}{a^3} \sum_n W_n \varepsilon_n, \quad p = \frac{1}{3a^3} \sum_n W_n p_{1,n},$$

the factor $1/3$ averaging a mode moving along x_1 over all directions. The program compares ρ and p with the kinetic-theory integrals.

(b) Pair creation. $a = e^t$ for $t < 0$ (de Sitter, $H_{\text{inf}} = 1$) joined smoothly to $a = (1 + 2t)^{1/2}$ for $t > 0$ (radiation); the integration is restarted at $t = 0$, where $\dot{H}$ jumps. Each mode starts in the vacuum when it is deep inside the horizon, at $t_0 = \ln(k/200)$ where $k/a = 200$, in the first-order adiabatic positive-frequency state, and runs until $H = 10^{-4}m$ (for $m = 0$ the $m = 0.1$ end is used). Masses $m \in \{0, 0.1, 0.5, 1, 2\}$, 64 Gauss-Legendre nodes in $\ln k$ on $[10^{-3}, 40]$. The number of created pairs per mode is $|\beta_k|^2 = |P_- u|^2 / |u|^2$, the weight on the negative-energy eigenspace ($P_- = (1 - h/E)/2$), and the comoving number density is $na^3 = \frac{16}{2\pi^2} \int k^2 |\beta_k|^2 dk$. For $m = 0$ the mode equation is conformally invariant and nothing is created.

6.15 EXP-5: momentum along an extra time

The extra times deflate, $c = e^{-t}$ ($H = 1$), and the mode has momentum q along x_5 and none elsewhere, $m = 1$. Then $k_5/h_5 = Q(t) = q e^t$ and

$$h = -im\gamma^4 - Q(t)\gamma^4\gamma^5, \quad E^2 = m^2 - q^2 e^{2t}, \quad h_i = -m\gamma^4, \quad h_r = -Q\gamma^4\gamma^5.$$

E^2 turns negative at $t^* = \ln(m/q)$. The state is the 32-real spinor; the initial state is the positive-energy eigenvector of $h(0)$, $E_0 = \sqrt{m^2 - q^2}$, that is also an eigenvector of C with eigenvalue ± 1 (so that its Krein norm $\pm E_0/m$ is not zero). Runs $q \in \{0.05, 0.1\}$ and $C = \pm 1$, t from 0

to $t^* + 3$, 601 samples. After t^* the Hilbert norm grows like $e^{2W(t)}$ with the WKB exponent $W(t) = \sqrt{Q^2 - m^2} - m \arccos(m/Q)$, while the Krein norm stays constant. This is why the quantisation and the cosmology of EXP-1 to EXP-4 are restricted to the good sector.

6.16 How CVODE solves $u' = f(t, u)$

Every experiment hands CVODE a system $Y' = f(t, Y)$ with a start value $Y(t_0)$ and a list of output times. CVODE advances in steps $t_{n+1} = t_n + h_n$ with a **linear multistep method**, which combines the last few values of Y and f :

- **Adams-Moulton** (orders 1 to 12) is accurate for smooth, non-stiff problems. Our mode equations are oscillations ($-ih$ has purely imaginary eigenvalues in the good sector), the classic non-stiff case.
- **BDF**, backward differentiation formulas (orders 1 to 5), is stable for stiff problems, which contain fast decaying components; it damps oscillations slightly.

Both formulas are implicit: Y_{n+1} appears on both sides, $Y_{n+1} = \gamma f(t_{n+1}, Y_{n+1}) + a_n$. CVODE solves this either by **Newton's method**, which needs the Jacobian $J = \partial f / \partial Y$ (here estimated by difference quotients and factorised as a dense matrix), or by **fixed-point iteration** $Y \leftarrow \gamma f(Y) + a_n$, which needs no Jacobian and converges when the step is small compared with the inverse rates, as it is for non-stiff problems.

After each step CVODE estimates the local error e_i of every component and accepts the step if the weighted root-mean-square $\sqrt{\frac{1}{n} \sum_i (e_i / (\text{rtol} |Y_i| + \text{atol}))^2}$ is at most 1; otherwise it retries with a smaller step. **rtol** is the relative tolerance, **atol** the absolute tolerance (important for components near zero), and **max_step** a cap on the step size. The tolerances bound the error of one step; the error at the end, the global error, accumulates and is larger (EXP-1 measured about $500 \times \text{rtol}$), which is why each experiment checks itself against exact solutions or conserved quantities.

Experiment	Method	rtol, atol	max_step	Steps	RHS calls
EXP-1	BDF + Newton + dense	1e-12, 1e-14	0.02	33866	34950
EXP-2	Adams + fixed point	1e-12, 1e-15	0.02	1779784	1781396
EXP-3	Adams + fixed point	1e-11, 1e-13	0.01	6074	10115
EXP-4	Adams + fixed point	1e-13, 1e-14	2.0	287898778	435564940
EXP-5	Adams + fixed point	1e-10, 1e-12	0.02	2548	4800

Why these settings, as measured by the study: EXP-1 at rtol 1e-10 drifts by 5e-8 to 1.2e-7 in its conserved quantities, above its 1e-8 limit (Exercise 13.1 lets you see it). EXP-2 without the step cap drifts by 1.7e-7 over $t \approx 10^4$. EXP-4 needs rtol 1e-13 because each thermal mode turns through about 10^5 radians and Adams leaks norm in proportion to steps times rtol; it chooses Adams over

BDF at run time by a measured test (Adams was more accurate, $1.7\text{e-}9$ against $1.8\text{e-}8$, and cheaper). For EXP-3 the step cap is in units of N .

7 Running the experiments

7.1 The command line of the program

```
dirac16complex_cosmology <print-config|exp1|exp2|exp3|exp4|exp5|all>
  [--output DIR]    output root (default artifacts/dirac16complex/numerics);
                    experiment N writes into DIR/expN/
  [--rtol X]        replace every experiment's default relative tolerance
  [--atol X]        replace every experiment's default absolute tolerance
  [--refined]       rtol/10, atol/10, max_step/2 (convergence runs)
```

Every self-check prints PASS - name: detail or FAIL - name: detail, and the **last line** is always SUCCESS (exit code 0) or FAILURE (exit code 1). The default output folder is relative to the folder you run the program from, so **always run it from the repository root**.

Save the long program path in a variable, which lasts until you close the terminal. PowerShell:

```
$bin = ".\studies\dirac16complex_cosmology\target\release\dirac16complex_cosmology.exe"
& $bin print-config
```

Git Bash, macOS and Linux (in Git Bash the .exe may be left out):

```
bin=./studies/dirac16complex_cosmology/target/release/dirac16complex_cosmology
$bin print-config
```

print-config runs nothing; it prints the configuration. Expected output (shortened):

```
study          = dirac16complex-cosmology
engine         = sundials_rs 7.8.0 CVODE (pure Rust, vendor/rustSolveIt)
fixture        = artifacts/dirac16complex/arbitrary-field/algebra-fixture.json
fixture sha256 = 8b4f15462ca4d61ef6ec0e72f04c8a77d23ce8bcabc9b6ce19f921c90e02653b
...
exp5: default rtol = 1.000000000000000004e-10, atol = 9.9999999999999980e-13, ...
all              = exp1, exp2, exp3, exp4, exp5
SUCCESS
```

The numbers are printed with 17 significant digits, the exact decimal value of the binary number, which is why 10^{-10} appears as $1.000000000000000004\text{e-}10$.

7.2 A first run into a scratch folder

Run EXP-1 into the scratch folder build/student (ignored by Git, so nothing committed is touched). PowerShell:

```
& $bin exp1 --output build/student
```

Git Bash, macOS and Linux:

```
$bin exp1 --output build/student
```

Expected output (0.2 seconds):

```
== exp1 ==
PASS - exact_solution_all_runs: max |u - exp(-iht)u0| = 6.24599767338994569e-09 ...
PASS - rho_frozen_all_runs: max |rho(t) - rho(0)| = 4.19369583504192178e-09
...
PASS - lambda_run_meff_constant: max |M_eff(t) - M*| = 1.32129440721939773e-09
exp1: solver_steps=33866 rhs_evaluations=34950 files=29 verdict=SUCCESS
SUCCESS
```

The files are now in `build/student/exp1/`. To see the exit code, type `$LASTEXITCODE` in PowerShell or `echo $?` in bash right after the run: 0 means success.

7.3 All five experiments

Run the others the same way (`exp2` to `exp5`), or all at once with `all`. What each prints at the end and how long it took on the test computer:

Experiment	Self-checks	Final summary line	Files	Time
exp1	10	exp1: solver_steps=33866 rhs_evaluations=34950 files=29 verdict=SUCCESS	29	0.2 s
exp2	15	exp2: solver_steps=1779784 rhs_evaluations=1781396 files=4 verdict=SUCCESS	4	5 s
exp3	14	exp3: solver_steps=6074 rhs_evaluations=10115 files=11 verdict=SUCCESS	11	0.1 s

Experiment	Self-checks	Final summary line	Files	Time
exp4	23	exp4: so lver_ste ps=287898778 rhs_evaluat ions=435564940 files=13 verdict= SUCCESS	13	72 s
exp5	7	exp5: so lver_ste ps=2548 rhs_eval uations=4800 files=5 ver dict=SUCCESS	5	0.1 s

That is 69 self-checks in total. EXP-4 integrates 288 million CVODE steps on up to 8 threads; on a computer with fewer cores expect several minutes. The line `== expN ==` starts each experiment and a single `SUCCESS` ends the whole run.

7.4 Reproducing the committed files byte for byte

The strongest check is to run everything into the default folder, over the committed files, and ask Git whether any byte changed. PowerShell:

```
& $bin all
git status --short
```

Git Bash, macOS and Linux:

```
$bin all
git status --short
```

The second command prints nothing if every file is identical to the committed one. That was the result on Windows 11 in both shells and on Ubuntu 24.04, all with the `win11` engine (80 to 90 seconds). With the `linux` or `macos` engine it lists the 14 files of Section 4.5. To put the committed files back after any experiment, in any shell:

```
git restore artifacts
```

8 Reading the output files

8.1 The file format

Every file is plain text. A CSV file has one header row of column names, then one row per sample, values separated by commas, lines ending in a single line feed. Every number is written with 18 significant digits in exponential form, for example $-2.10000000000113253\text{e}+01$, so that a repeat run can be compared byte for byte; a value that does not exist is written `nan`. Each experiment folder also contains `summary.json` with the parameters, tolerances, solver statistics, every self-check and the verdict. Open the files with any text editor or a spreadsheet, or read them with Python (Section 13 shows how).

8.2 EXP-1 (folder `exp1`, 29 files)

`background_A1.csv` and `background_A2.csv` (201 rows each) describe the field for the two profiles:

Column	Meaning
<code>t</code>	time $t = Hx_4$
<code>a4, a4_prime, a4_second</code>	the profile a_4 and its first and second derivatives
<code>scale_3space, scale_extretime</code>	e^{a_4} and e^{-a_4}
<code>volume_ratio</code>	V/V_0 (equal to 1)
<code>theta</code>	total expansion rate Θ (equal to 0)
<code>rho_req</code>	energy density that 8D Einstein gravity would need
<code>p_req_0, p_req_1, p_req_2, p_req_3, p_req_5, p_req_6, p_req_7</code>	required pressures along the seven transverse directions
<code>p_mean_req, w_req</code>	their mean and the required w

The 26 run files `run_A{1,2}_K{0,0p5,2}_{pos_Bp,pos_Bm,neg_Bp,mix}.csv` and `run_A{1,2}_K0p5_pos_Bp_lambda0p5.csv` (the letter p stands for the decimal point) have 201 rows each:

Column	Meaning
<code>t</code>	time
<code>u_re_0 to u_re_15, u_im_0 to u_im_15</code>	the state: real and imaginary parts of u
<code>S</code>	scalar density $S = S_0 s(u)$
<code>M_eff</code>	effective mass
<code>energy_mode</code>	$\varepsilon(u) = u^\dagger h u$
<code>rho</code>	energy density
<code>p_0, p_1, p_2, p_3, p_5, p_6, p_7</code>	pressures along the seven transverse directions
<code>p_mean, w</code>	mean pressure and $w = \bar{p}/\rho$
<code>KE_L, PE_L, KE_H, PE_H</code>	the two kinetic/potential splits
<code>w_L</code>	$(KE_L - PE_L)/\rho$
<code>norm_hilbert, norm_krein</code>	$u^\dagger u$ and $u^\dagger B u$
<code>exact_error</code>	distance from the exact solution

8.3 EXP-2 (folder exp2, 4 files)

run_x0_0.csv, run_x0_m0p4.csv and run_x0_0p5.csv (511 rows each, in increasing t : the backward branch, $t = 0$, the forward branch; m stands for minus):

Column	Meaning
t	time
ln_b, ln_a, ln_c	logarithms of the scale factors (state 0 to 2)
H_b, H_a, H_c	expansion rates (state 3 to 5)
u_re_0 to u_re_15, u_im_0 to u_im_15	the spinor (state 6 to 37)
V, Theta	7-volume and total expansion rate
s_u, S, M_eff	$s(u)$, the condensate, the effective mass
energy_mode	$u^\dagger h u$
rho, p, w	energy density, isotropic pressure, w
KE_L, PE_L, KE_H, PE_H	the two splits
constraint_residual, constraint_relative	$\sum_{i<j} H_i H_j - \kappa \rho$, absolute and divided by the sum of the absolute terms
bound	$\Theta^2 - 3H_a^2 - 2\kappa \rho$ (equal to $H_b^2 + 3H_c^2 \geq 0$ on the constraint surface)
w_eff	the w a 3-space observer infers, $-1 + \frac{\Theta}{3H_a}(1 + w)$
norm_hilbert, norm_krein	the two norms

8.4 EXP-3 (folder exp3, 10 run files and the analysis)

run_x0_{0,m0p2,m0p3,m0p433107,m0p462654}_mu{3,7}.csv, the backward and forward branches joined in increasing N :

Column	Meaning
N, a, z	e-folds, scale factor, redshift $z = 1/a - 1$
branch	-1 backward, 0 today, +1 forward
H0t, D_C	state 0 and 1: $H_0(t - t_0)$ and the comoving distance
u_re_0 to u_re_15, u_im_0 to u_im_15	state 2 to 33: the spinor
ln_sigma	state 34
E	H/H_0
q	deceleration parameter
rho_psi, p_psi, w	condensate density, pressure, w
KE_L, PE_L, KE_H, PE_H	the two splits
Omega_psi, Omega_m	density fractions ρ_ψ/E^2 and $\Omega_m a^{-3}/E^2$ at that time
cs2	sound speed squared $dp/d\rho = 2x_0\sigma/(1 + 2x_0\sigma)$
M_eff_over_H0, s_u	M_{eff}/H_0 and $s(u)$
sigma_spinor, sigma_state, sigma_analytic	σ from the spinor, from state 34, and a^{-3}
norm_hilbert, norm_krein	the two norms
eigen_leak	how far u has left the rest eigenspace
rho_closed, p_closed, w_closed	the closed forms of Section 6.13

Column	Meaning
d_L, mu_H0free	luminosity distance $(1+z)D_C$ and $5 \log_{10} d_L$ (nan for $z \leq 0$)

The analysis writes `fits.json` (all fits and the Unite comparison), `fits_mu_scan.csv` (distance-modulus fits for x_0 from -0.49 to 0 in steps of 0.01) and `fits_scan.csv` (closed forms and $w(a)$ fits in steps of 0.001):

Column of <code>fits_scan.csv</code>	Meaning
x0	interaction parameter
w0_tangent, wa_tangent	CPL tangent at $a = 1$
a_zero, z_zero	where $\rho_\psi = 0$: $a = x_0 ^{1/3}$
a_cross, z_cross	where $w = -1$ is crossed: $a = (2 x_0)^{1/3}$
a_bounce, z_bounce	the bounce $E^2 = 0$
q0, cs2_today	deceleration and sound speed squared today
wfit_requested_defined, wfit_requested_w0, wfit_requested_wa	the least-squares CPL fit of $w(a)$ on $[1/3.26, 1]$ (1 if defined, else 0 and nan)
wfit_restricted_a_lo, wfit_restricted_w0, wfit_restricted_wa	the same fit restricted to where $w \geq -3$

8.5 EXP-4 (folder `exp4`, 13 files)

File	Rows	Contents
<code>thermal_grid.csv</code>	48	the momentum grid: <code>node</code> , Gauss-Legendre <code>x</code> and <code>gl_weight</code> on $[-1, 1]$, <code>k</code> , <code>weight</code> on $[0, 120]$, <code>E_i</code> , occupation <code>f</code> , <code>mode_weight</code> W_n , <code>beta2_sudden_start</code> (the predicted free-wave amplitude)
<code>thermal_modes.csv</code>	2928	every thermal mode at 61 times
<code>thermal_spin.csv</code>	244	a second spin state for four nodes (must equal the first)
<code>thermal_antiparticle.csv</code>	61	a negative-energy mode (antiparticle symmetry)
<code>thermal_adiabatic_vacuum.csv</code>	122	two modes started in the first-order adiabatic vacuum

File	Rows	Contents
thermal_eos.csv	61	the gas: a, t, H, rho, p, w, KE_H, PE_H, KE_L, PE_L, n_a3, the kinetic-theory rho_kinetic, p_kinetic, w_kinetic, the deviations rel_dev_rho, rel_dev_p, and beta2_weighted, beta2_max, beta2_adiabatic_max, norm_dev_max
pair_grid.csv	64	node, x, gl_weight, k, ln_k_weight, start time t0
pair_modes.csv	2560	every pair-creation mode at 8 times, for the five masses
pair_antiparticle.csv	24	negative-energy modes for $m = 1$
pair_spectrum.csv	320	m, node, k, t0, a_end, $ \beta_k ^2$ at the start, at the joint $t = 0$ and at the end (beta2_initial, beta2_kink, beta2_adiabatic_kink, beta2_end, beta2_adiabatic_end) and the high- k theory beta2_kink_tail_theory
pair_history.csv	35	m, sample, t, a, n_a3, n_a3_adiabatic
pair_eos.csv	305	the created gas: m, a, n_a3, rho_a3, p_a3, w

The mode files share these columns:

Column	Meaning
m (pair files only), node, k	mass, grid node, comoving momentum
t, a, H, K	time, scale factor, expansion rate, physical momentum k/a
u_re_0 to u_re_15, u_im_0 to u_im_15	the state
E, eps, p1	$\sqrt{m^2 + K^2}$, $\varepsilon(u)$, $p_1(u)$
s (thermal files only)	$s(u)$
norm_hilbert, norm_krein	the two norms
beta2, beta2_adiabatic	$ \beta_k ^2$ in the instantaneous and in the first-order adiabatic basis

8.6 EXP-5 (folder exp5, 5 files)

run_q0p05_Cp.csv, run_q0p05_Cm.csv, run_q0p1_Cp.csv, run_q0p1_Cm.csv (601 rows each; Cp and Cm are $C = +1$ and $C = -1$):

Column	Meaning
t	time
Q	$q e^t$
E2	$E^2 = m^2 - Q^2$
kappa	WKB growth rate $\sqrt{Q^2 - m^2}$ (0 before t^* ; not the gravitational coupling)
wkb_W	WKB exponent $W(t)$
u_re_0 to u_re_15, u_im_0 to u_im_15	the state
norm_hilbert, norm_krein, ln_norm_hilbert	the norms and $\ln(u^\dagger u)$
krein_drift, krein_drift_normalized	change of $u^\dagger B u$, absolute and divided by $\max(u^\dagger u, 1)$

9 Checking the results with the Python checkers

9.1 What the checkers do

Each checker `scripts/check_dirac16complex_expN.py` uses numpy and the standard library only. It rebuilds the gamma matrices from the algebra fixture, recomputes every physical quantity from the state columns alone (never trusting the program's derived columns), compares with exact solutions and independent integrators, checks conservation laws and finite-difference residuals of the equations, and compares with the program's `summary.json`. Two options add two more checks:

```
--repeat DIR    run the program again into DIR; every file must be byte-identical
--refined DIR    run it again with ten times tighter tolerances; the errors must shrink
```

The checker prints `check_NAME=true` or `false` for each check, then `check_count=` and `failed_checks_count=`, writes `python-check-report.json` into the experiment folder, and exits with 1 if any check failed.

9.2 Checking the committed folder (the full form)

After Section 7.4, run the EXP-3 analysis first (the EXP-3 checker also checks its `fits.json`), then the five checkers with both options, in any shell:

```
python scripts/analyze_dirac16complex_exp3.py
python scripts/check_dirac16complex_exp1.py --repeat build/repeat --refined build/refined
python scripts/check_dirac16complex_exp2.py --repeat build/repeat --refined build/refined
python scripts/check_dirac16complex_exp3.py --repeat build/repeat --refined build/refined
python scripts/check_dirac16complex_exp4.py --repeat build/repeat --refined build/refined
python scripts/check_dirac16complex_exp5.py --repeat build/repeat --refined build/refined
git status --short
```

Expected results and times on the test computer:

Script	Checks	Failed	Time
analyze_dirac16c omplex_exp3.py	9	0	4 s
check_dirac16com plex_exp1.py	25	0	2 s
check_dirac16com plex_exp2.py	34	0	19 s
check_dirac16com plex_exp3.py	31	0	1 s
check_dirac16com plex_exp4.py	51	0	3 to 4 min
check_dirac16com plex_exp5.py	21	0	5 s

That is $25 + 34 + 31 + 51 + 21 = 162$ checker checks. The EXP-4 checker is slow because it runs EXP-4 twice more and integrates reference modes with its own Runge-Kutta method. On Windows the final Git command again printed nothing: the checker reports and the analysis files are byte-identical too. On Ubuntu (Python 3.12.3, numpy 2.5.3) all checks passed as well, but the five reports and the three analysis files differed in their last digits, because Python's mathematical functions and numpy come from different libraries there; that is expected.

9.3 Checking a scratch folder (the quick form)

For your own runs into `build/student` (Section 7.2), point the checker at that folder with its output option; without the repeat and refined options two checks are skipped. In any shell:

```
python scripts/analyze_dirac16complex_exp3.py --output build/student
python scripts/check_dirac16complex_exp1.py --output build/student
```

and likewise for exp2 to exp5. The quick form gives 23, 32, 29, 49 and 19 checks for EXP-1 to EXP-5. If you run the EXP-3 checker before the analysis, `fits.json` is missing and it reports one check fewer (28 in the quick form, 30 in the full form).

10 The Jupyter notebook

10.1 What it is

`notebooks/dirac16complex_dark_sector.ipynb` is a Python 3 notebook of 23 cells, 8 of them code. For each experiment it runs the program into the scratch folder `build/notebook-run`, checks that every file is byte-identical to the committed one, recomputes the key physics from the raw spinor columns, draws 17 figures into `artifacts/dirac16complex/numerics/figures/`, and ends with a gauntlet of 70 assertions. It finds the program through the environment variable `DIRAC16_BIN` or at the release path of Section 5.1, so build the program first. Its driver is adapted, with attribution, from the rustSolveIt engine's `planet_Mercury/notebook` (Section 16).

10.2 Running it headless

The standard-library runner executes every cell in order and writes the outputs back into the notebook only if every cell succeeds. In any shell:

```
python notebooks/run_notebook.py notebooks/dirac16complex_dark_sector.ipynb
```

It prints the notebook's output while it runs (about 100 seconds) and ends with

```
PASS - prose_numbers_match_reports: 64 numbers quoted in the markdown re-read ...
ALL CHECKS PASSED
ok notebooks/dirac16complex_dark_sector.ipynb (8 cells)
1 ok, 0 failed
```

(on Windows the paths in these lines are printed with backslashes). Jupyter's own executor writes an executed copy instead. PowerShell:

```
$nb = "notebooks/dirac16complex_dark_sector.ipynb"
$opts = "--output-dir", "build/nbconvert", "--ExecutePreprocessor.timeout=3600"
python -m nbconvert --to notebook --execute $nb @opts
```

Git Bash, macOS and Linux:

```
python -m nbconvert --to notebook --execute notebooks/dirac16complex_dark_sector.ipynb \
    --output-dir build/nbconvert --ExecutePreprocessor.timeout=3600
```

nbconvert comes with JupyterLab (Section 3.5). Then audit both executed copies. PowerShell:

```
$copy = "build/nbconvert/dirac16complex_dark_sector.ipynb"
python notebooks/check_notebook.py $nb --also $copy
```

Git Bash, macOS and Linux:

```
python notebooks/check_notebook.py notebooks/dirac16complex_dark_sector.ipynb \
    --also build/nbconvert/dirac16complex_dark_sector.ipynb
```

Expected output:

```
ok notebooks/dirac16complex_dark_sector.ipynb: all structure and execution rules
pass (23 cells, 8 code cells, 70 gauntlet checks, 17 figures)
ok build/nbconvert/dirac16complex_dark_sector.ipynb: all structure and execution
rules pass (23 cells, 8 code cells, 70 gauntlet checks, 17 figures)
ok cross-check: both executions printed identical gauntlet results and figure hashes
```

(each result is one line on your screen). On Windows, the Git check of Section 7.4 still printed nothing afterwards: the executed notebook and the 17 figures were byte-identical to the committed ones. On Ubuntu the gauntlet and the audit passed as well, but the figures and the notebook differed in their bytes (a different matplotlib build draws the same curves with different pixels); `git restore artifacts notebooks` puts them back. The nbconvert route may print a `RuntimeWarning` about the Proactor event loop on Windows; it is harmless.

10.3 Running it interactively

With JupyterLab installed, in any shell:

```
python -m jupyterlab notebooks/dirac16complex_dark_sector.ipynb
```

A browser tab opens with the notebook. If you are asked for a kernel, choose Python 3 (ipykernel). Click the first cell and press Shift+Enter repeatedly (each press runs one cell and moves to the next), or use the menu Run, Run All Cells. If no browser opens, start it like this instead and open the printed address, which begins with `http://localhost:8888/lab?token=`, yourself:

```
python -m jupyterlab --no-browser notebooks/dirac16complex_dark_sector.ipynb
```

To stop JupyterLab, press Ctrl+C twice in the terminal. The notebook is generated by `notebooks/build_dirac16complex_notebook.py`: change the builder, not the notebook, and after rebuilding run `run_notebook.py` again, because the builder writes an unexecuted notebook.

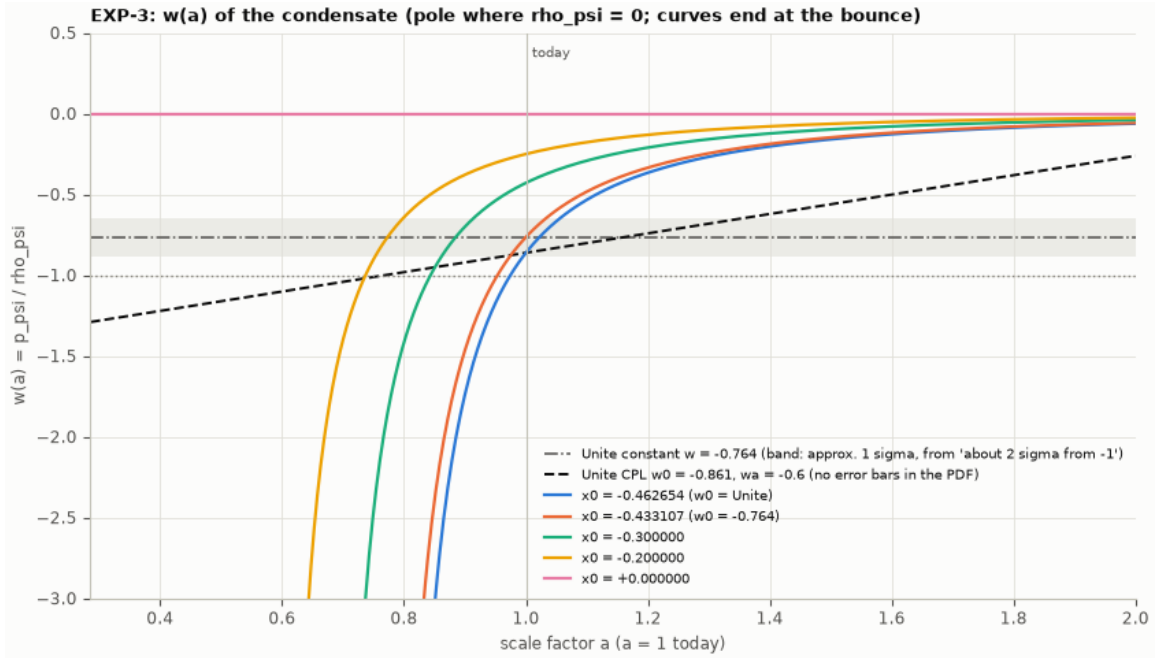


Figure 1: The EXP-3 equation of state $w(a)$ for the five values of x_0 , with the Unite CPL line and the constant $w = -0.764$ band, as the notebook draws it

11 The Mathematica notebook (optional)

`notebooks/Dirac16ComplexDarkSector.nb` is an independent cross-check in the Wolfram Language. It loads the exact packages `wolfram/Dirac16ComplexAlgebra.wl` and `wolfram/Dirac16ComplexGeometry.wl`, derives each experiment's reduced system symbolically from the covariant field equation and compares it with the program's equations, re-integrates each experiment with `NDSolve` and

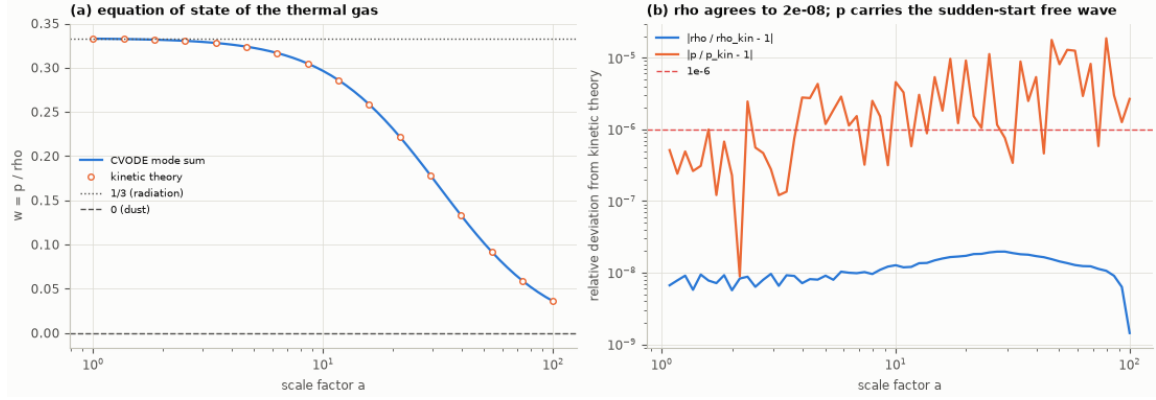


Figure 2: EXP-4: the thermal gas goes from $w = 1/3$ (radiation) towards $w = 0$ (dust) as the universe expands; the CVOICE mode sum agrees with kinetic theory

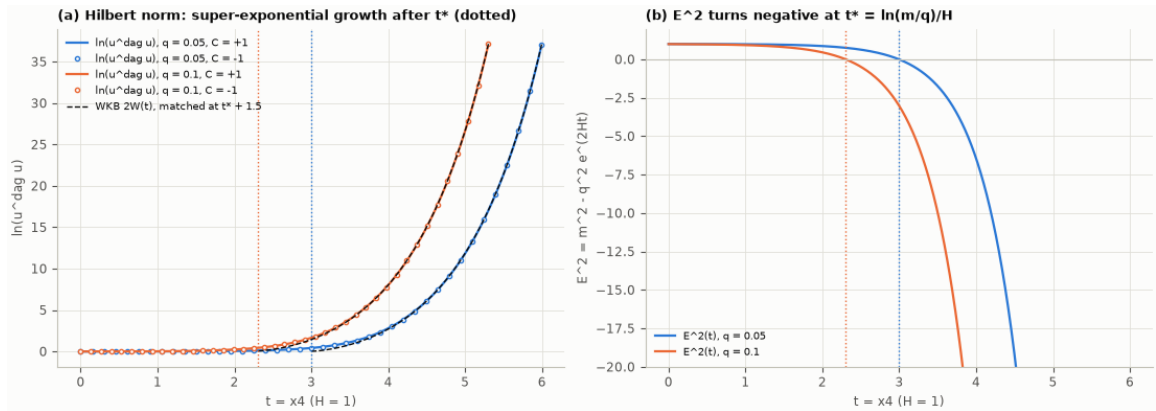


Figure 3: EXP-5: after t^* the Hilbert norm grows super-exponentially, following the WKB exponent $2W(t)$, while E^2 is negative

compares with the committed CSV columns, runs the program's `print-config` through `RunProcess`, draws 8 figures into `artifacts/dirac16complex/numerics/figures/mathematica/` and ends with 49 checks. It needs the built program (Section 5).

It is new work: none of the three `rustSolveIt` repositories contains a Mathematica notebook or package (no `.nb`, `.wl` or `.wls` file; each holds 294 Jupyter notebooks), which was verified by listing their full Git trees. It is modelled on the notebook `DiracTriality.nb` of the `dirac` repository (<https://github.com/once-ere/dirac>) and its build and verify scripts.

Evaluate it headless (about 4 to 5 minutes), in any shell:

```
wolframscript -file scripts/verify_dirac16complex_mathematica_notebook.wls
```

Expected output at the end:

```
input_cell_count=37
failed_evaluation_count=0
message_count=0
notebook_check_count=49
failed_notebook_check_count=0
...
dirac16complex_mathematica_notebook=OK
```

It rewrites `artifacts/dirac16complex/numerics/mathematica-report.json` and the 8 figures; on Windows the Git check of Section 7.4 printed nothing afterwards. The notebook itself is generated by

```
wolframscript -file scripts/build_dirac16complex_mathematica_notebook.wls
```

which writes the same bytes again. To work with it interactively, open the `.nb` file in Mathematica (the free Wolfram Engine has no notebook window) and choose Evaluation, Evaluate Notebook; it finds the repository from the notebook's own location. The interactive route was not tested for this guide.

12 Rebuilding the PDF documents (optional)

Every document in `provenance/` is written in Markdown and converted to LaTeX and PDF by one command, which runs the converter twice, `pdflatex` three times into two fresh folders, requires warning-free logs and two byte-identical PDFs, and compares the PDF with the edition registered in `provenance/pdf-specifications.json`. For this guide, in any shell:

```
python scripts/build_provenance_pdf.py provenance/DIRAC16COMPLEX_STUDENT_GUIDE.md
```

It prints one `check_NAME=true` line per check, then `check_count=14`, `failed_check_count=0` and `provenance_pdf=OK`. The other documents are built the same way, for example `provenance/DIRAC`

16COMPLEX_PRIMORDIAL_FIELD.md. `pdflatex` always runs with the repository root as its working folder, because the figure paths are relative to it.

The registered PDF bytes belong to the TeX installation that registered them (MiKTeX 26.5 on Windows). With another TeX installation the document still builds warning-free and byte-identically twice, but the check `registeredSha256` is false, the PDF is not copied, and the command exits with 1; on Ubuntu with TeX Live 2023 exactly this happened (12 of 14 checks true). The builder's register option, which records a new edition in the registry, is for the maintainer of a document only.

13 Changing parameters: exercises with answers

The physical parameters are constants at the top of each file `studies/dirac16complex_cosmology/src/expN.rs`; the command line changes only the tolerances and the output folder. Always send changed runs into a scratch folder with the output option, as below, so the committed files stay intact. After editing a source file, rebuild (Section 5.1); to undo the edit, run `git restore` with the file name and rebuild again.

13.1 Tolerances decide whether the checks pass

Exercise. Run EXP-1 with `rtol` 10^{-10} instead of 10^{-12} . Predict which self-checks fail.

PowerShell:

```
& $bin exp1 --output build/ex-rtol --rtol 1e-10
```

Git Bash, macOS and Linux:

```
$bin exp1 --output build/ex-rtol --rtol 1e-10
```

Answer. The run needs fewer steps (`solver_steps=18458`) and the conserved quantities drift more: `rho_frozen_all_runs` ($1.2e-7$), `pressures_and_S_frozen_eigenstate_runs`, `hilbert_norm_conserved` and `krein_norm_conserved` ($5.4e-8$) and `lambda_run_meff_constant` ($1.4e-8$) FAIL against their $1e-8$ limits, while `exact_solution_all_runs` ($7.8e-8$ against $1e-7$) still passes. The last line is **FAILURE** and the exit code is 1. The limits were set from the physics, and the default tolerance was tightened until they hold, never the other way round.

13.2 Convergence

Exercise. Run EXP-1 with the refined option (Section 7.1) into `build/ex-refined`. By how much does the distance from the exact solution shrink?

PowerShell:

```
& $bin exp1 --output build/ex-refined --refined
```

Git Bash, macOS and Linux:

```
$bin exp1 --output build/ex-refined --refined
```

Answer. The tolerances become $\text{rtol } 1\text{e-}13$, $\text{atol } 1\text{e-}15$ and $\text{max_step } 0.01$, the step count grows to 57960, and the largest distance from the exact solution (maxExactError in `summary.json`) falls from $6.2\text{e-}9$ to $1.3\text{e-}10$: the solution converges.

13.3 Predicting EXP-1 from the formulas

Exercise. For the positive-energy eigenmode with $K = 2$ and $\lambda = 0$, predict ε , ρ , p_0 , $\bar{p}$, w , KE_L and PE_L , then read them from `run_A1_K2_pos_Bp.csv`. Hint: on an eigenmode $\varepsilon = E$ and $p_0 = K^2/E$, and only p_0 is nonzero.

The Python lines below read the first and last row. PowerShell:

```
@'
import csv
path = "artifacts/dirac16complex/numerics/exp1/run_A1_K2_pos_Bp.csv"
rows = list(csv.DictReader(open(path)))
for key in ("energy_mode", "rho", "p_0", "p_mean", "w", "KE_L", "PE_L"):
    print(key, float(rows[0][key]), float(rows[-1][key]))
'@ | python -
```

Git Bash, macOS and Linux:

```
python - <<'EOF'
import csv
path = "artifacts/dirac16complex/numerics/exp1/run_A1_K2_pos_Bp.csv"
rows = list(csv.DictReader(open(path)))
for key in ("energy_mode", "rho", "p_0", "p_mean", "w", "KE_L", "PE_L"):
    print(key, float(rows[0][key]), float(rows[-1][key]))
EOF
```

Answer. $E = \sqrt{1+4} = 2.23607 = \varepsilon = \rho$; $p_0 = K^2/E = 1.78885$; $\bar{p} = p_0/7 = 0.25555$; $w = K^2/(7E^2) = 4/35 = 0.11429$; $KE_L = PE_L = E/2 = 1.11803$. The first and last rows agree to about 10^{-9} : the field is frozen. Note that $KE_L = PE_L$ although $w \neq 0$: for a moving mode the scalar-field relation $p = KE_L - PE_L$ does not hold (Section 6.10).

13.4 Solving the EXP-2 constraint yourself

Exercise. Open `studies/dirac16complex_cosmology/src/exp2.rs` in a text editor (Notepad, TextEdit, nano or any code editor) and change the line `pub const HUBBLE_C0: f64 = -0.2;` to `-0.3;` save the file. Predict C_0 , Θ_0 , S_0 for $x_0 = 0$ and the time at which H_c changes sign. Then rebuild (Section 5.1) and run, in PowerShell:

```
& $bin exp2 --output build/ex-hc
```

or in Git Bash, macOS and Linux:

```
$bin exp2 --output build/ex-hc
```

Answer. $C_0 = 3 + 3(0.09) + 9(-0.3) = 0.57$, $\Theta_0 = 3 - 0.9 = 2.1$, $S_0 = 0.57$ (and 0.95, 0.38 for $x_0 = -0.4, 0.5$), $\alpha = 7S_0/12 = 0.3325$; $H_c V = H_{c0} + \beta t$ with $\beta = S_0/6 = 0.095$ changes sign at $t = 0.3/0.095 = 3.158$. The run gives these values in `summary.json` (keys `S0`, `constraintLhs0`, `theta0`, `alpha`, `extraTimeTurnTime`), and 14 of the 15 self-checks pass. `late_time_isotropic_dust` FAILS ($1.17e-3$ against its $1e-3$ limit): with the larger initial anisotropy, the fixed end point $V/V_0 = e^{18.5}$ is no longer late enough for the anisotropy to fall below the limit. A self-check with an a-priori limit tests the physics of the chosen parameters, not only the solver. Undo the edit with `git restore studies/dirac16complex_cosmology/src/exp2.rs` and rebuild.

13.5 From x_0 to (w_0, w_a)

Exercise. For $x_0 = -0.25$ predict w_0 , w_a , the redshift where $\rho_\psi = 0$ and the redshift of the phantom crossing $w = -1$. Then read the fine scan of the analysis.

PowerShell:

```
@'
import csv
path = "artifacts/dirac16complex/numerics/exp3/fits_scan.csv"
rows = list(csv.DictReader(open(path)))
row = [r for r in rows if abs(float(r["x0"]) + 0.25) < 1e-9][0]
for key in ("w0_tangent", "wa_tangent", "z_zero", "z_cross", "z_bounce"):
    print(key, float(row[key]))
'@ | python -
```

Git Bash, macOS and Linux:

```
python - <<'EOF'
import csv
path = "artifacts/dirac16complex/numerics/exp3/fits_scan.csv"
rows = list(csv.DictReader(open(path)))
row = [r for r in rows if abs(float(r["x0"]) + 0.25) < 1e-9][0]
for key in ("w0_tangent", "wa_tangent", "z_zero", "z_cross", "z_bounce"):
    print(key, float(row[key]))
EOF
```

Answer. $w_0 = x_0/(1 + x_0) = -1/3$; $w_a = 3x_0/(1 + x_0)^2 = -4/3$; $\rho_\psi = 0$ at $a = |x_0|^{1/3} = 0.62996$, i.e. $z = 1/a - 1 = 0.58740$; $w = -1$ where $\rho + p = 0$, at $a = (2|x_0|)^{1/3} = 0.79370$, $z = 0.25992$. The script prints -0.3333 , -1.3333 , 0.58740 , 0.25992 and the bounce $z_b = 0.74542$, which needs the cubic of Section 6.13.

Exercise. Which x_0 gives $w_0 = -0.9$, and what is its w_a ? **Answer.** $x_0 = w_0/(1 - w_0) = -0.47368$ and $w_a = -5.13$: the closer w_0 is to -1 , the faster the condensate evolves, the opposite of the slowly varying Unite fit ($w_a = -0.60$). The scan row $x_0 = -0.474$ gives $w_0 = -0.90114$, $w_a = -5.1396$.

13.6 Moving the EXP-5 turning point

Exercise. In `studies/dirac16complex_cosmology/src/exp5.rs` change the line `pub const Q_VALUES: [f64; 2] = [0.05, 0.1];` to `[0.05, 0.2]` and save. Predict t^* and the end time for $q = 0.2$. Then rebuild and run, in PowerShell:

```
& $bin exp5 --output build/ex-q
```

or in Git Bash, macOS and Linux:

```
$bin exp5 --output build/ex-q
```

Answer. $t^* = \ln(1/0.2) = \ln 5 = 1.60944$ and $t_{\text{end}} = t^* + 3 = 4.60944$. All 7 self-checks pass; the new files are `run_q0p2_Cp.csv` and `run_q0p2_Cm.csv`; `summary.json` gives `tStar` 1.6094379124341003, a final $u^\dagger u$ of 1.32×10^{16} and WKB deviations of $1.0\text{e-}3$ (leading order) and $2.4\text{e-}4$ (first order). The EXP-5 Python checker compares the parameters with the study's specification and reports that mismatch, as it should. Undo with `git restore studies/dirac16complex_cosmology/src/exp5.rs` and rebuild.

13.7 Massless quanta are not created

Exercise. Before looking, say what na^3 EXP-4 should find for $m = 0$. **Answer.** Zero: a massless spinor's mode equation is conformally invariant, and a radiation or de Sitter universe is conformally flat. The run finds $na^3 = 1.8 \times 10^{-24}$ for $m = 0$, pure roundoff, against 1.45×10^{-3} for $m = 0.1$ and 4.99×10^{-3} for $m = 0.5$ (`pair.masses` in `exp4/summary.json`, column `n_a3` of `pair_history.csv`).

14 Troubleshooting

Symptom	Cause	What to do
LNK1104: cannot open file ...\\deps\\...rllib while linking on Windows	the path is longer than 260 characters	clone into a short folder such as <code>\$HOME\\src</code> (Section 4.1)
error: linker link.exe not found on Windows	the Microsoft C++ build tools are missing	install them (Section 3.2) and open a new terminal
error: linker cc not found on Linux, or a message on macOS that the command line developer tools are missing	no C linker	install <code>build-essential</code> (Section 3.4) or the Xcode command line tools (Section 3.3)
<code>rustc</code> or <code>cargo</code> not found right after installing	the terminal was opened before the installation	open a new terminal; on macOS and Linux also <code>source "\$HOME/.cargo/env"</code>

Symptom	Cause	What to do
the program stops at once with an illegal-instruction error (STATUS_ILLEGAL_INSTRUCTION on Windows, SIGILL elsewhere)	the processor has no FMA, which <code>.cargo/config.toml</code> requires (Section 5.2)	use a computer with an x86-64 processor from 2013 or later
error: failed to load manifest for dependency ccode_rs, caused by failed to read ...vendor/rustSolveIt/.../Cargo.toml	the engine was not fetched	run the setup script (Section 4.3)
vendor/rustSolveIt is at ..., expected ...	the folder holds another engine	delete it and run the setup script again (Section 4.4)
cargo prints Blocking waiting for file lock	another cargo command is running	wait for it to finish
error: failed to remove file ...dirac16complex_cosmology.exe, caused by Access is denied. (os error 5), on Windows	the program is still running (or an antivirus or synchronisation tool holds the file)	wait until the run has finished, then build again; work outside synchronised folders
running scripts is disabled on this system	Windows PowerShell 5.1 blocks scripts	use <code>pwsh</code> , or <code>powershell -ExecutionPolicy Bypass -File ...</code> (Section 4.3)
UnicodeEncodeError: 'charmap' codec can't encode character when Python output is redirected to a file or pipe on Windows	Python writes redirected output in the code page cp1252	make Python use UTF-8 for the session: <code>\$env:PYTHONUTF8 = "1"</code> (PowerShell) or <code>export PYTHONUTF8=1</code> (Git Bash)
jupyter or another script is not recognized, and pip warned that its folder is not on PATH	pip installed it into the per-user Scripts folder	use <code>python -m jupyterlab</code> and <code>python -m nbconvert</code> , as this guide does
error: external managed-environment from pip	macOS or Linux protects its system Python	use the virtual environment of Section 3.5
No module named numpy in a new terminal on macOS or Linux	the virtual environment is not active	<code>source ~/.venvs/dirac16/bin/activate</code>
No module named nbconvert	JupyterLab is not installed	<code>python -m pip install jupyterlab</code>
the Git check of Section 7.4 lists 14 changed files after all	the linux or macos engine is in use (Section 4.5)	fetch the win11 engine and rebuild; <code>git restore artifacts</code> restores the files
the notebook stops with <code>AssertionError('fresh_outputs_by te_identical')</code>	the same cause	the same remedy
the EXP-3 checker reports 30 checks instead of 31	it ran before the analysis	run <code>analyze_dirac16complex_exp3.py</code> first

Symptom	Cause	What to do
wolframscript is not recognized	WolframScript is not installed or not on PATH	install the Wolfram Engine or Mathematica; on Windows add its folder for the session with <code>\$env:Path += ";C:\Program Files\Wolfram Research\WolframScript"</code>
wolframscript asks for a Wolfram ID or reports that the kernel is not activated	the Wolfram Engine was never activated	run <code>wolframscript -activate</code> yourself once
the PDF build lists latex_warning= lines and fails the check logWarningFree	a LaTeX package is missing or a line is too wide	read build/NAME/pdf-a/NAME.log; let MiKTeX install missing packages, or install texlive-latex-recommended and lmodern on Linux
the PDF build fails only registeredSha256 and provenancePdfCopy	a different TeX installation made different bytes	expected (Section 12); the PDF was still built in build/NAME/pdf-a/

15 Glossary

- a : the scale factor of 3-space ($a = 1$ today); in EXP-2 the common scale factor of x_1, x_2, x_3 . b , c : the scale factors of the hidden space x_0 and of the extra times x_5, x_6, x_7 .
- **Adams-Moulton**: a family of implicit multistep methods for non-stiff ODEs (Section 6.16).
- **atol, rtol**: absolute and relative error tolerances of CVODE.
- $B = -iC\gamma^4$: the Hermitian matrix that defines the Krein inner product $u^\dagger Bu$.
- **BDF**: backward differentiation formulas, implicit multistep methods for stiff ODEs.
- $|\beta_k|^2$: the weight of a mode on the negative-energy states: the number of created particle-antiparticle pairs per mode.
- **bounce**: the moment at which $E^2 = H^2/H_0^2$ reaches 0 in EXP-3, so that the universe stops contracting backward in time.
- **byte-identical**: two files with exactly the same bytes; Git reports no change.
- $C = \gamma^0\gamma^1\gamma^2\gamma^3$: the charge matrix; $\bar{\Psi} = \Psi^\dagger C$.
- **cargo, rustc, rustup**: Rust's build tool, compiler and toolchain installer.
- **chirality**: the eigenvalue ∓ 1 of γ^8 ; components 0 to 7 have -1 , 8 to 15 have $+1$.
- **CPL**: the parametrisation $w(a) = w_0 + w_a(1 - a)$, so $w_a = -dw/da$.
- c_s^2 : the sound speed squared $dp/d\rho$; negative means the condensate's perturbations grow.
- **CSV**: comma-separated values, the output format of the program.

- **CVODE**: the variable-step, variable-order ODE integrator of SUNDIALS.
- D_C, d_L : comoving distance and luminosity distance $(1+z)D_C$, in units of c/H_0 .
- **dark energy**: the component with negative pressure ($w < -1/3$) that accelerates the expansion.
- **dark matter**: pressureless ($w \approx 0$) matter that does not emit light.
- E : the mode energy, $h^2 = E^2$; in EXP-3 instead the dimensionless expansion rate H/H_0 .
- **effective mass** $M_{\text{eff}} = m + \lambda S$: the mass shifted by the self-interaction.
- $\varepsilon(u) = u^\dagger h u$: the energy of one mode.
- **engine**: the rustSolveIt folder `sundials_rs` with the crates `sundials_core` and `cvode_rs`.
- **equation of state parameter** $w = p/\rho$.
- η : the flat metric $\text{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$.
- **expectation-value rule**: $\langle \Psi^\dagger M \Psi \rangle = u^\dagger B M u$ for a one-particle state on the normalised mode u .
- **FMA**: fused multiply-add, $a \cdot b + c$ with one rounding (Section 5.2).
- **fixed-point iteration, Newton's method**: the two ways CVODE solves the implicit equation of each step.
- **freezing, thawing**: dark-energy fields whose w falls towards -1 (freezing, $w_a > 0$) or rises from -1 (thawing, $w_a < 0$) as the universe expands, with the formula-consistent signs of Section 1.1.
- **gamma matrices** γ^a : the eight 16 by 16 matrices with $\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab}$ (Section 6.3).
- **good sector**: modes without momentum along the extra times; there h is Hermitian.
- **Grassmann-odd**: anticommuting, $\Psi_a \Psi_b = -\Psi_b \Psi_a$.
- $h(t)$: the mode Hamiltonian, $i\dot{u} = hu$ (Section 6.8). h_j : the scale factor of direction j (a different symbol, always with an index).
- H_j, Θ, V : expansion rate of direction j , their sum, and the 7-volume.
- H_0 : today's expansion rate; H_{inf} : the de Sitter rate of EXP-4(b); H_i : the initial rate 0.05 of EXP-4(a).
- **Hilbert norm** $u^\dagger u$, **Krein norm** $u^\dagger B u$.
- K : the physical momentum (k/a in EXP-4, the hidden-space momentum in EXP-1). k : comoving momentum. $q, Q = qe^t$: extra-time momentum (EXP-5).
- κ : the gravitational coupling $8\pi G$; in EXP-5 the column `kappa` is instead the WKB rate.
- KE_L, PE_L, KE_H, PE_H : the Lagrangian and Hamiltonian kinetic/potential splits (Section 6.10).

- λ : the four-fermion coupling; $U(S) = \frac{\lambda}{2}S^2$.
- **mode**: a plane-wave solution $\Psi = V^{-1/2}e^{ik \cdot x}u(t)$.
- μ : in EXP-3 the reduced frequency $M_{\text{eff}}(a=1)/H_0$; the distance modulus is written `mu_H0fre` in the CSV.
- $N = \ln a$: e-folds, the independent variable of EXP-3.
- $\Omega_r, \Omega_m, \Omega_\psi$: today's density fractions of radiation, matter and the condensate.
- **phantom**: $w < -1$ with $\rho > 0$; for the condensate it happens exactly when $KE_L < 0$.
- **RHS**: the right-hand side $f(t, Y)$ of $Y' = f(t, Y)$.
- $\rho, p, \bar{p}$: energy density, pressure, mean transverse pressure.
- ρ_{req} : the energy density that 8D Einstein gravity would need to produce the primordial field (EXP-1).
- $S = \bar{\Psi}\Psi$: the scalar density (condensate); $s(u) = u^\dagger(-i\gamma^4)u$ per mode; $\sigma = S/S_0$.
- **scale factor**: the factor $h_j(t)$ that turns coordinate distance into physical distance.
- **spinor connection** Ω_μ , **covariant derivative** D_μ (Section 6.6).
- **state vector**: the real numbers CVODE integrates, laid out as in Sections 6.9 to 6.15.
- **SUNDIALS**: the SUite of Nonlinear and Differential/ALgebraic equation Solvers.
- $t^* = \ln(m/q)$: the time after which $E^2 < 0$ in EXP-5.
- **Unite**: the supernova compilation quoted in the input document ($w_0 = -0.861$, $w_a = -0.60$, constant $w = -0.764$).
- **vielbein**: the frame e^a that turns the curved metric into η .
- w_0, w_a : the CPL parameters; w_{eff} : the w a 3-space observer infers in EXP-2.
- **WKB**: the approximation $u^\dagger u \propto e^{2W}$ with $W = \int \sqrt{Q^2 - m^2} dt$ for a slowly varying rate.
- x_0 : the hidden space coordinate, and in EXP-2 and EXP-3 also the parameter $\lambda S_0/(2m)$; the meaning is always clear from the context.
- z : the redshift $1/a - 1$; in the primordial field of EXP-1 the variable $6Hx_0$.

16 Credits, licences and verified facts

- **This repository**: GPL-3.0-or-later (see LICENSE); original notebook and physical programme: Patrick L. Nash; publication tooling derived from <https://github.com/once-ere/dirac>. See NOTICE.

- **The engine:** the pure-Rust SUNDIALS 7.8.0 port of the rustSolveIt repositories, BSD-3-Clause as declared in their Cargo manifests (published by the once-ere account; LLNL copyright for SUNDIALS); its deterministic library contains translations of GNU C Library 2.39 under LGPL-2.1-or-later (NOTICE, section 2c). It is fetched by the setup scripts, never stored in this repository.
- **The Jupyter notebook tools** `notebooks/run_notebook.py`, `notebooks/check_notebook.py` and the builder pattern are adapted, with attribution in their headers, from `planet_Mercury/notebook` of rustSolveIt (the `md()` and `code()` helpers, `find_binary()` and `run()` with the rule that the last printed line must be `SUCCESS`, a `gauntlet()` of assertions, the standard-library runner and the auditor).
- **The Mathematica notebook** is new (Section 11).
- **Verified while writing this guide:** the pinned commits and the notebook counts of the three engine repositories (294 Jupyter notebooks and no `.nb`, `.wl` or `.wls` file in each); the differences between the engines (Section 4.5); the byte-for-byte reproduction of all Rust outputs with the `win11` engine on Windows 11 and on Ubuntu 24.04; and every expected output quoted above.
- **Not established by this guide:** anything about macOS, and anything physical beyond the committed results. The scientific conclusions, with their assumptions and limits, are in `DIRAC16COMPLEX_DARK_SECTOR_NUMERICS`.